

Taller Bintec 2026 — Guía paso a paso del flujo en n8n

Esta guía es para quien **nunca ha usado n8n**. Arma el mismo flujo de 15 nodos de la guía del taller ([Taller_Bintec_Guia_Completa.pdf](#)), pero clic a clic y con capturas, adaptado a este repositorio. Cada paso dice qué buscar, qué escribir y qué deberías ver al ejecutar.

Video: el mismo proceso, de principio a fin, en [VIDEO-PASO-A-PASO.mp4](#) (10:46). Capítulos: 0:00 Webhook · 1:05 Preparar payload · 1:59 Risk, Fraud y CRM · 3:50 Merge y Combinar respuestas · 4:40 Gemini Agent · 5:46 Parsear, auditoría y guardrail · 7:03 If, revisión humana y respuesta · 9:08 Probar y publicar.

⚠ **No copies código desde este PDF.** Al pegarlo en n8n, los saltos de línea del PDF quedan dentro del código o de la expresión y el nodo se muestra con errores (texto en rojo, *SyntaxError*, *Unexpected token*), aunque se vea igual. Copia los valores largos (código, bodies JSON, prompt de Gemini) desde la carpeta [snippets/](#) del repositorio, o desde la [versión web de esta guía](#) con el botón de copiar de cada bloque. En VS Code o Codespaces: abre el archivo, `Ctrl+A`, `Ctrl+C`. Cada paso indica qué archivo usar; el [README de snippets/](#) tiene todos los valores en tablas.

Diferencia con el PDF: aquí la URL y la llave del modelo van en el archivo `.env`. Los pasos 5 y 6 del PDF (crear la API key y la credencial de n8n) **no hacen falta**.

Antes de empezar, una advertencia: n8n trae un **asistente de IA** (el botón  de la derecha, "Ask n8n Assistant", "Build an agent", "Connect a model"). **En este taller no se usa.** El flujo se construye a mano, nodo por nodo; el modelo de lenguaje solo se llama desde **un** nodo (paso 8), y la llave del taller únicamente funciona ahí.

Puedes trabajar **con Docker en tu equipo** o, si no puedes instalarlo, **en GitHub Codespaces** desde el navegador (sección 1). El resto de la guía es igual en los dos casos.

Contenido

0. Conceptos mínimos de n8n
1. Preparar el entorno — con Docker o en Codespaces
2. Entrar a n8n
3. Crear el workflow
4. Paso 1 — Webhook
5. Paso 2 — Preparar payload
6. Pasos 3, 4 y 5 — Risk, Fraud y CRM en paralelo
7. Paso 6 — Merge
8. Paso 7 — Combinar respuestas
9. Paso 8 — Gemini Agent (el modelo)
10. Pasos 9, 10 y 11 — Parsear, auditar y calcular el guardrail

11. Paso 12 — If: ¿necesita revisión humana?
12. Pasos 13 y 14 — Rama de revisión humana
13. Paso 15 — Respond to Webhook
14. Probar el flujo completo
15. Publicar y probar con el script
16. Problemas comunes

0. Conceptos mínimos de n8n

Concepto	Qué es
Workflow	El flujo completo: una cadena de nodos conectados en el lienzo (<i>canvas</i>).
Nodo	Un paso del flujo: recibir una petición, llamar una API, ejecutar código, decidir...
Trigger	El primer nodo, el que arranca el flujo. Aquí es un Webhook (una URL que recibe un POST).
Item	Cada dato que viaja entre nodos es un objeto JSON. En este flujo siempre viaja 1 item .
Expresión <code>{{ }}</code>	Texto que n8n calcula al ejecutar. <code>{{ \$json.customer_id }}</code> = el campo <code>customer_id</code> del item que entra al nodo. Los campos en modo <i>Expression</i> se ven en verde.
<code>\$('Nombre')</code>	Lee el resultado de otro nodo por su nombre. Por eso algunos nodos deben llamarse exactamente como dice esta guía.
Ejecución de prueba	Mientras construyes, ejecutas el flujo desde el editor y ves los datos de cada nodo.
Publicar	Deja el flujo activo en su URL de producción (<code>/webhook/...</code>) sin necesidad del editor.

El panel de un nodo. Al hacer doble clic en un nodo se abre su panel, que tiene tres columnas: **INPUT** (lo que recibe del nodo anterior), **Parameters** (su configuración) y **OUTPUT** (lo que produjo en la última ejecución). Trabajar con datos reales en INPUT es lo que hace fácil construir: por eso en el paso 1 enviamos una solicitud de prueba antes de seguir.

Cómo agregar nodos.

- El `+` que aparece a la derecha de un nodo agrega el siguiente nodo conectado a su salida.
- Para conectar dos nodos que ya existen, **arrastra** desde el círculo de salida (derecha) de uno hasta el círculo de entrada (izquierda) del otro.
- Para sacar **varias ramas** de la misma salida, arrastra desde su círculo de salida y suelta en un espacio vacío del lienzo: se abre el buscador de nodos y el nuevo nodo queda conectado.
- Para **renombrar** un nodo, haz clic en su título dentro del panel (arriba a la izquierda), o selecciónalo en el lienzo y presiona `F2`.

n8n guarda los cambios automáticamente.

1. Preparar el entorno

Necesitas la **URL y la llave del modelo**: las del correo del taller (`LLM_BASE_URL` y `LLM_API_KEY`; la llave empieza por `sk-bintec-`) o tu propia API key de Gemini:

	<code>LLM_BASE_URL</code>	<code>LLM_API_KEY</code>
Llave del taller	la URL del correo	<code>sk-bintec-...</code>
Tu propia llave de Gemini	<code>https://generativelanguage.googleapis.com</code>	tu API key de Google AI Studio

Opción A: con Docker en tu equipo

Requisitos: Docker Desktop (o Docker Engine con Compose v2) en ejecución, y Git.

```
git clone https://github.com/juan-manjarr/taller-n8n-bintec-2026.git
cd taller-n8n-bintec-2026
cp .env.example .env          # PowerShell: Copy-Item .env.example .env
```

Abre `.env` y completa las dos líneas:

```
LLM_BASE_URL=<la URL>
LLM_API_KEY=<tu llave>
```

Levanta el entorno y espera a que `n8n` aparezca como `healthy`:

```
docker compose up -d
docker compose ps
```

`n8n` queda en <http://localhost:5678>. Los servicios simulados tienen su documentación en <http://localhost:8001/docs>, <http://localhost:8002/docs> y <http://localhost:8003/docs>.

Opción B: en GitHub Codespaces (sin Docker)

Codespaces crea una máquina en la nube de GitHub con el mismo entorno; lo usas desde el navegador. Solo necesitas una cuenta de GitHub.

1. Abre <https://github.com/juan-manjarr/taller-n8n-bintec-2026> y haz clic en **Code** → **Codespaces** → **Create codespace on main**.
2. Se abre un editor (VS Code en el navegador). La primera vez tarda 3–5 minutos porque descarga las imágenes. Cuando termina, la terminal de abajo muestra *"Imágenes descargadas. Falta un paso"*.
3. En el explorador de archivos (izquierda) abre `.env` (ya viene creado) y completa `LLM_BASE_URL` y `LLM_API_KEY`. Guarda con `Ctrl+S`.
4. En la terminal ejecuta:

```
docker compose up -d
```

5. Abre la pestaña **PORTS** (junto a *TERMINAL*), busca la fila **n8n (5678)** y haz clic en el ícono del globo. n8n se abre en una pestaña nueva con una URL `https://...-5678.app.github.dev`.

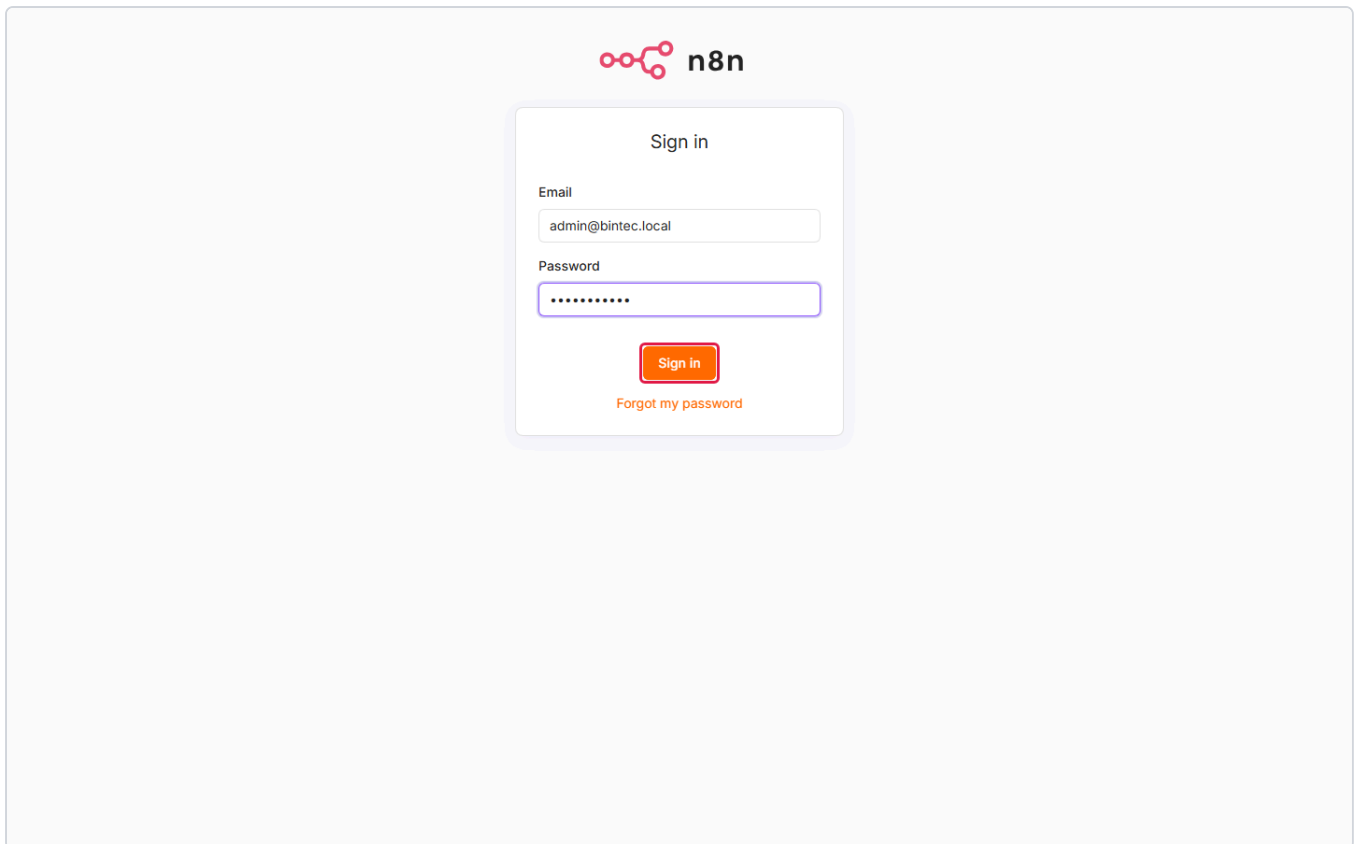
Diferencias con la opción A durante el resto de la guía:

- Donde la guía dice `http://localhost:5678` en el navegador, usa la URL `...app.github.dev`.
- Los comandos de prueba se ejecutan **en la terminal del codespace** y son los de **bash** (`./scripts/test-flow.sh ...`), no los de PowerShell. Las URLs de webhook que muestra n8n (`http://localhost:5678/...`) funcionan desde esa terminal.
- La extensión de **Postman** para VS Code viene instalada en el codespace (ícono de Postman en la barra lateral; pide iniciar sesión con una cuenta de Postman). Corre dentro del codespace, así que usa `http://localhost:5678`: por ejemplo `POST http://localhost:5678/webhook-test/solicitud-bancaria` con *Body* → *raw* → *JSON* y el contenido de un archivo de `requests/`.
- **No cambies la visibilidad del puerto a *Public***: cualquiera con la URL podría entrar con la contraseña del taller.
- El codespace se suspende tras 30 minutos sin uso. Al reabrirlo (<https://github.com/codespaces>) los servicios se levantan solos y tu flujo sigue ahí.
- Al terminar el taller, elimina el codespace en <https://github.com/codespaces> para no gastar tu cuota gratuita.

Si cambias `.env` después (en cualquiera de las dos opciones), ejecuta otra vez `docker compose up -d` para que n8n tome los valores nuevos.

2. Entrar a n8n

Abre <http://localhost:5678> e inicia sesión con `admin@bintec.local` / `Bintec2026!`.

The image shows the n8n web interface's sign-in page. At the top center is the n8n logo, which consists of a red icon of three connected circles followed by the text 'n8n'. Below the logo is a white rectangular sign-in form with a light gray border. The form has a title 'Sign in' at the top. It contains two input fields: 'Email' with the value 'admin@bintec.local' and 'Password' with masked characters. Below the password field is an orange 'Sign in' button. At the bottom of the form is a link that says 'Forgot my password'.

Al entrar puede aparecer la ventana **Build and debug faster with the n8n Assistant**. Haz clic en **Set up later in Settings** (no en *Get started*): el taller no usa el asistente de IA y la llave del taller no funciona con él.



Build and debug faster with the n8n Assistant

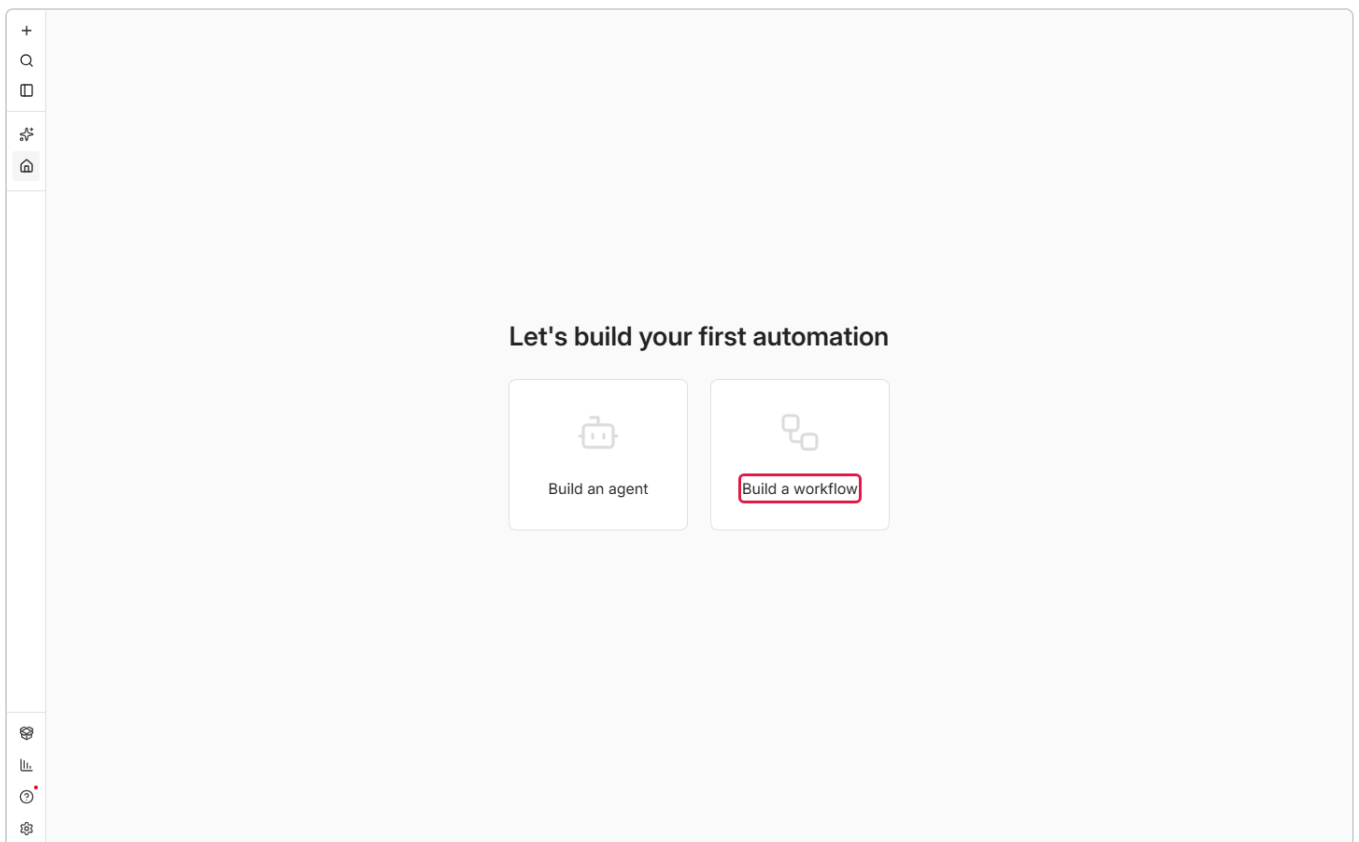
Preview

- 🔗 Build and edit workflows through conversation
- 🧪 Debug failed executions and suggest fixes
- 🕒 Ask anything about n8n and get contextual help

Get started

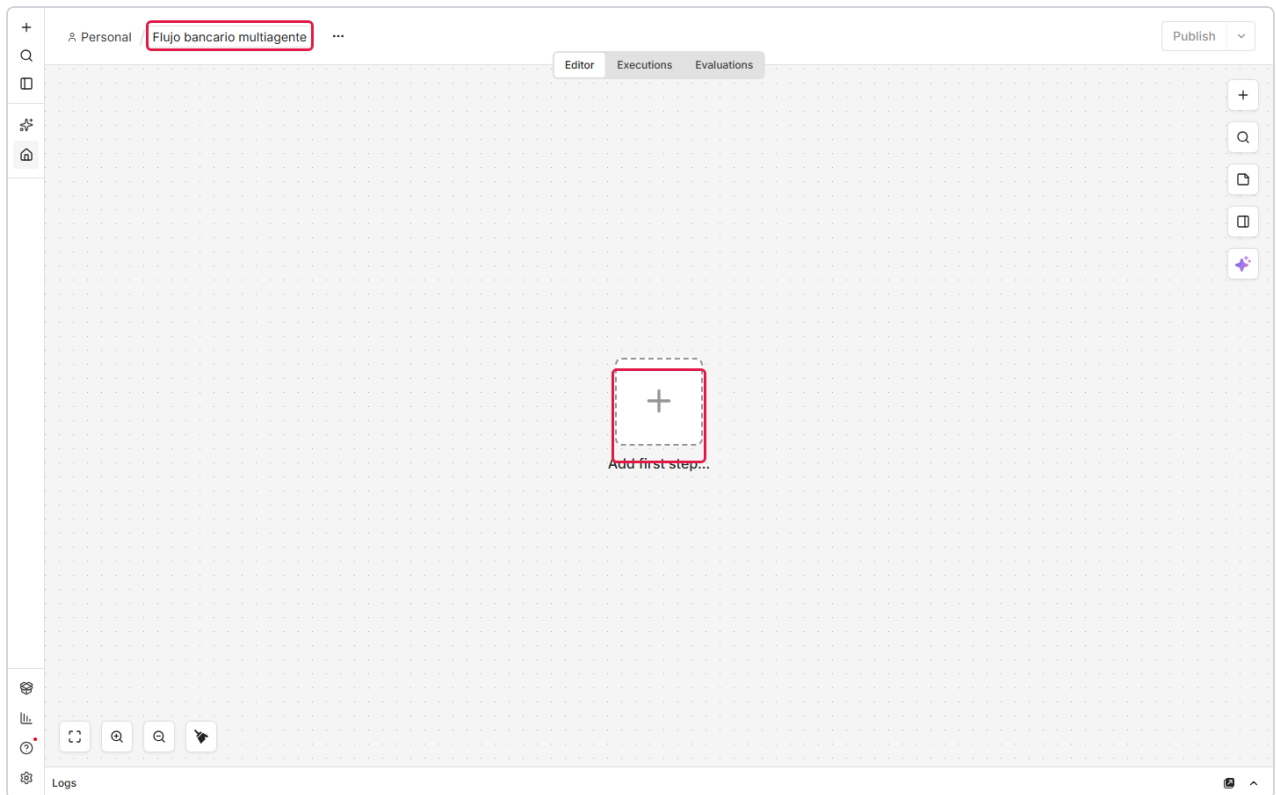
Set up later in Settings

Verás la pantalla de inicio. **No uses "Build an agent"** (es el asistente de IA): el taller usa **Build a workflow**.



3. Crear el workflow

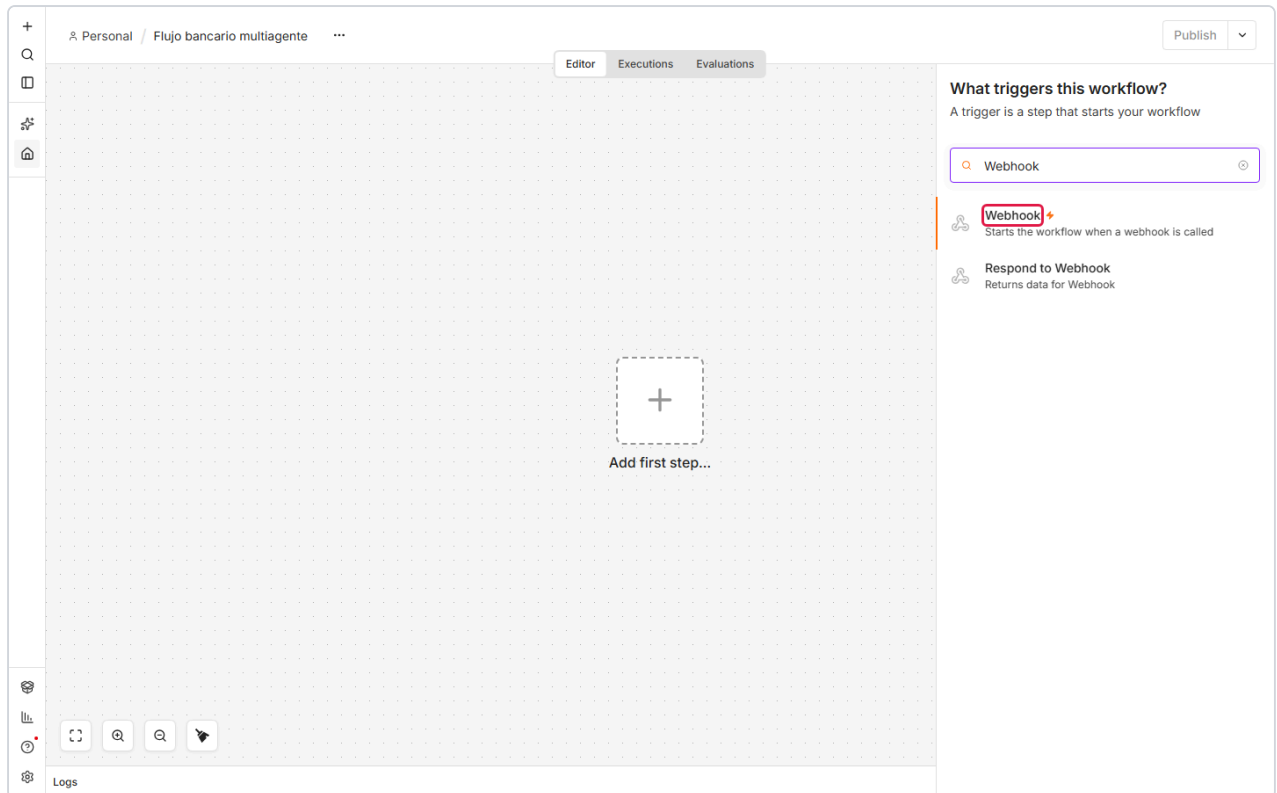
1. En la pantalla de inicio haz clic en **Build a workflow** (si ya creaste otro flujo antes, usa el botón **Create workflow** de arriba a la derecha).
2. Haz clic en el nombre *My workflow* (arriba a la izquierda) y cámbialo por **Flujo bancario multiagente**. El lienzo vacío muestra **Add first step....**



4. Paso 1 — Webhook

El webhook es la puerta de entrada: una URL que recibe la solicitud bancaria por POST.

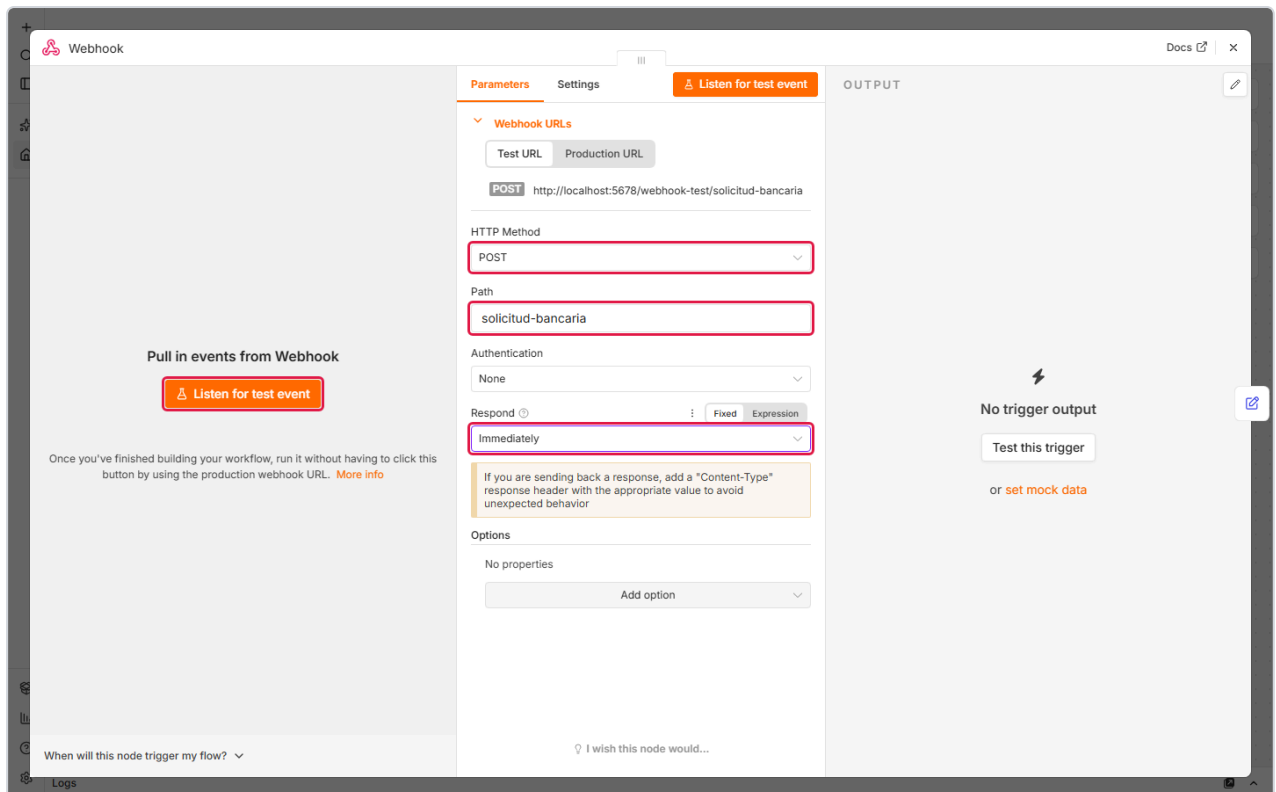
1. Haz clic en **Add first step...**, escribe `Webhook` y presiona `Enter`.



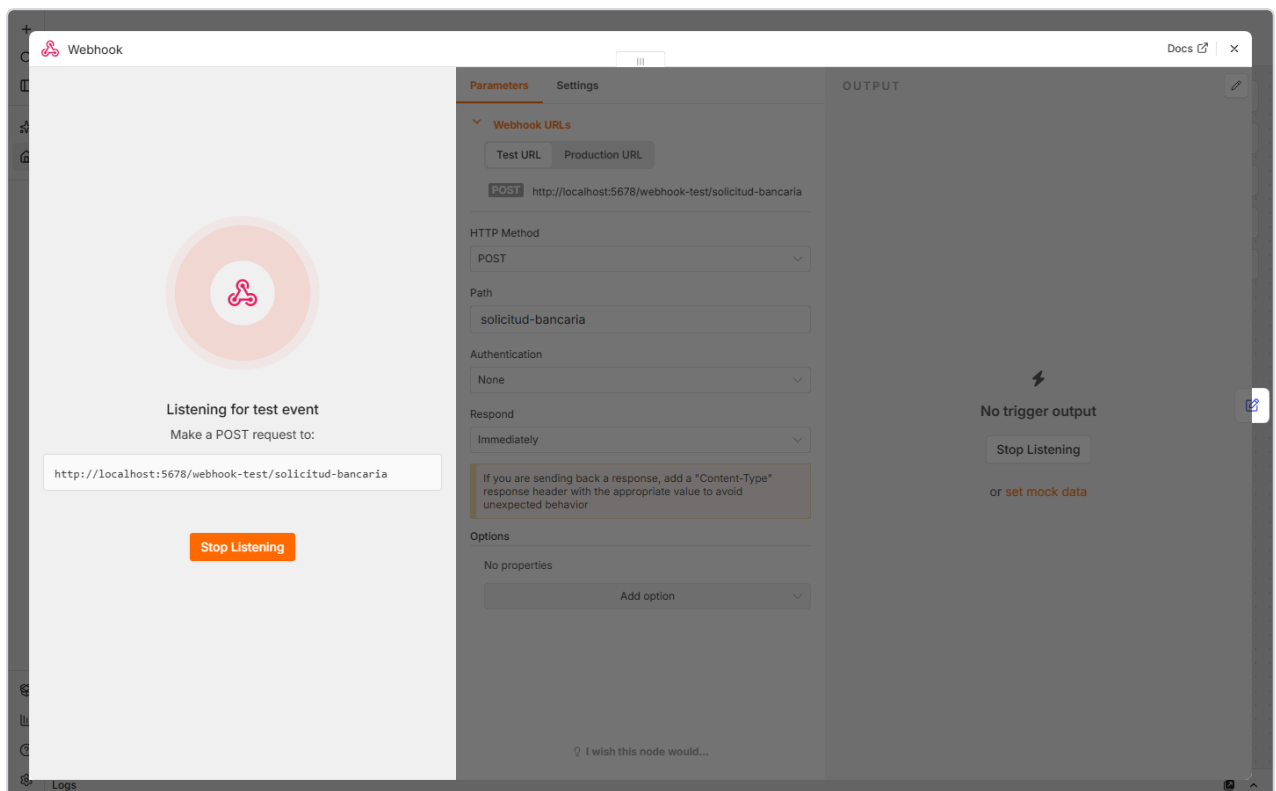
2. Configura el nodo:

Campo	Valor
HTTP Method	POST
Path	solicitud-bancaria
Authentication	None
Respond	Immediately (por ahora; en el paso 15 lo cambiamos)

Por qué Immediately y no Using 'Respond to Webhook' Node como dice el PDF: con esa opción n8n exige que el nodo *Respond to Webhook* ya exista, y la prueba falla con "No Respond to Webhook node found in the workflow". Lo cambiamos al final, cuando ese nodo exista.



3. **Envía una solicitud de prueba** para tener datos reales con qué trabajar. Haz clic en **Listen for test event**. El nodo queda esperando:



Ahora, **desde una terminal** en la carpeta del repositorio (en Codespaces, la terminal del codespace con el comando de bash), envía el primer caso de ejemplo:

```
.\scripts\test-flow.ps1 -Test -Files requests\01-cliente-preferencial.json
```

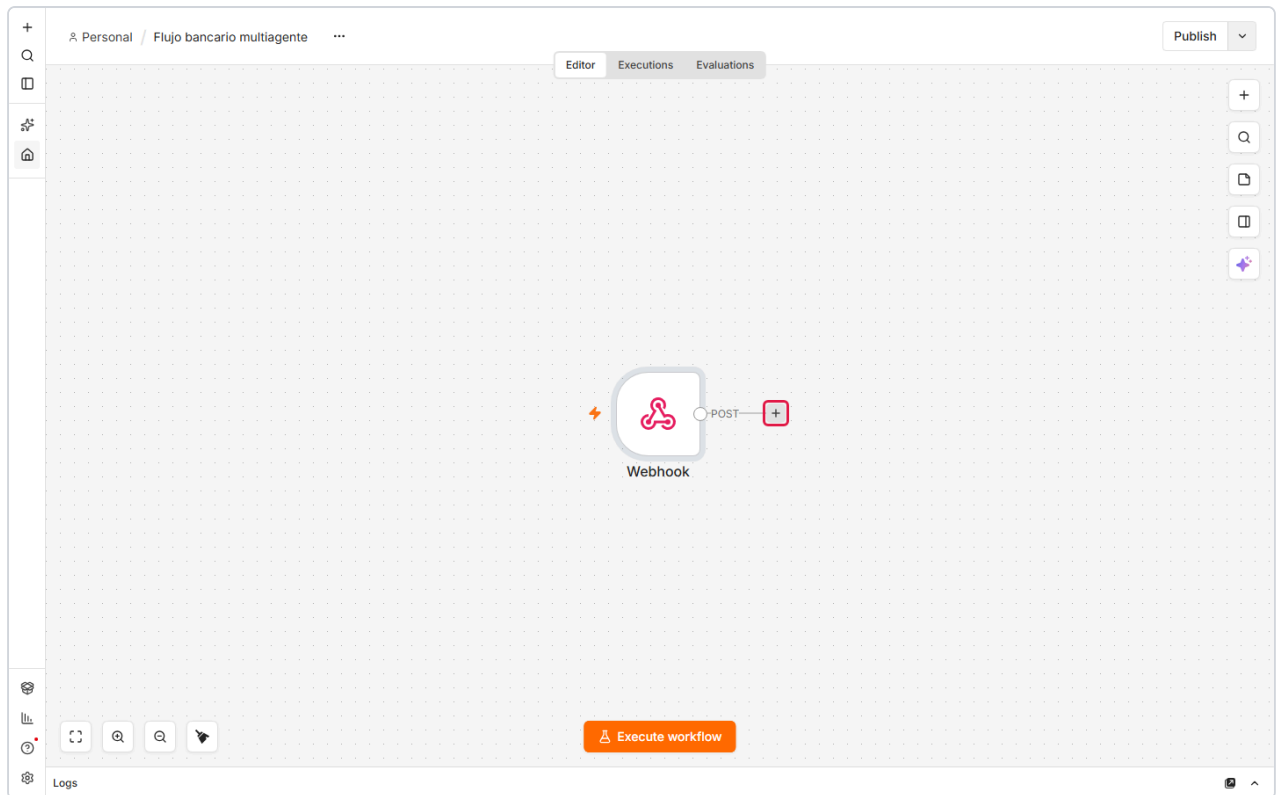
```
WEBHOOK_PATH=webhook-test ./scripts/test-flow.sh requests/01-cliente-preferencial.json
```

En OUTPUT aparecen los datos recibidos; la solicitud viene dentro de **body** :

The screenshot shows the n8n Webhook node configuration interface. On the left, there's a 'Pull in events from Webhook' section with a 'Listen for test event' button. The main configuration area is divided into 'Parameters' and 'Settings' tabs. Under 'Parameters', the 'Webhook URLs' section is expanded, showing a 'Test URL' and a 'Production URL'. The 'Test URL' is set to 'http://localhost:5678/webhook-test/solicitud-bancaria'. The 'HTTP Method' is set to 'POST'. The 'Path' is set to 'solicitud-bancaria'. The 'Authentication' is set to 'None'. The 'Respond' section is set to 'Immediately'. There's a note: 'If you are sending back a response, add a "Content-Type" response header with the appropriate value to avoid unexpected behavior'. The 'Options' section is set to 'No properties'. On the right, the 'OUTPUT' panel shows the received data in a table format. The table has columns for 'headers', 'parameters', and 'body'. The 'headers' column contains: 'host : localhost:5679', 'connection : keep-alive', 'content-type : application/json', 'accept : */*', 'accept-language : *', 'sec-fetch-mode : cors', 'user-agent : node', 'accept-encoding : gzip, deflate', and 'content-length : 208'. The 'parameters' column contains two '(empty object)' entries. The 'body' column contains a JSON object with fields: 'customer_id', 'customer_name', 'requested_loan_amount', 'term_months', 'channel', 'ip', and 'device_id'. A tooltip says: 'You can pin this output instead of waiting for a test event.' At the bottom right, a green checkmark indicates 'Node executed successfully'.

4. **Recomendado — fija (pin) estos datos.** Haz clic en el ícono de pin (📌) arriba a la derecha de OUTPUT. Así, cada vez que ejecutes, n8n reutiliza esta solicitud y no tienes que volver a enviarla. Antes de la prueba final (sección 14) quítale el pin.

5. Cierra el panel con la **X**. En el lienzo, el nodo muestra un **+** a su derecha.

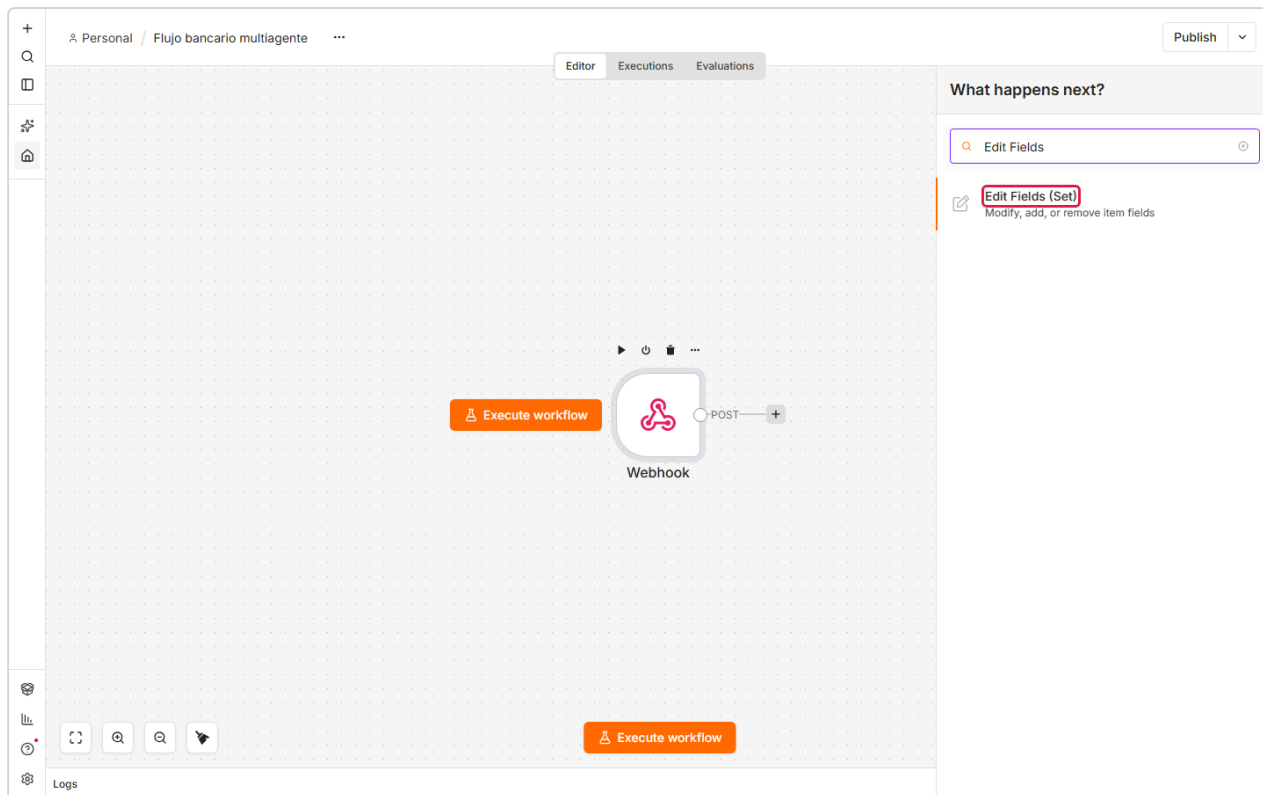


La URL de prueba atiende una sola solicitud por clic. Si envías y obtienes "*The requested webhook ... is not registered*", vuelve a hacer clic en *Listen for test event* (o en *Execute workflow*) y reenvía.

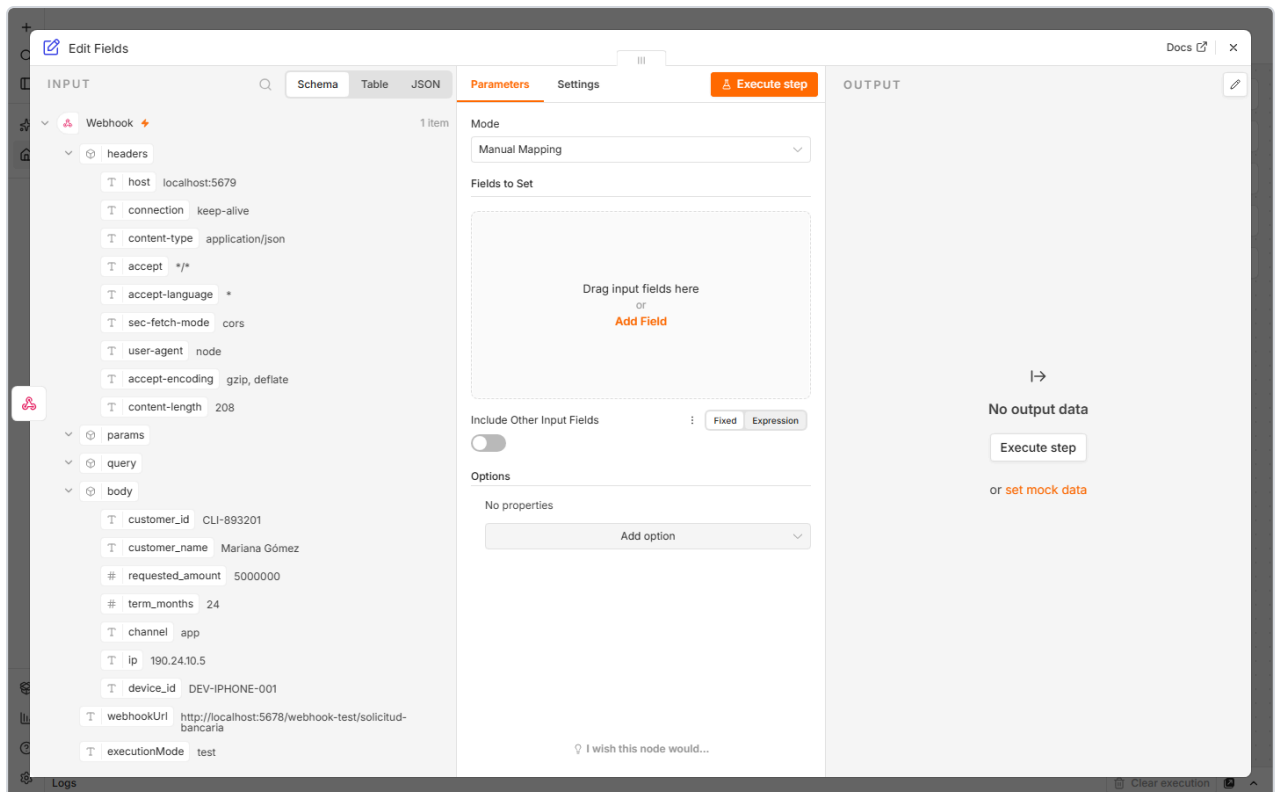
5. Paso 2 — Preparar payload

Este nodo saca los campos de `body` y los deja "planos" para los siguientes pasos.

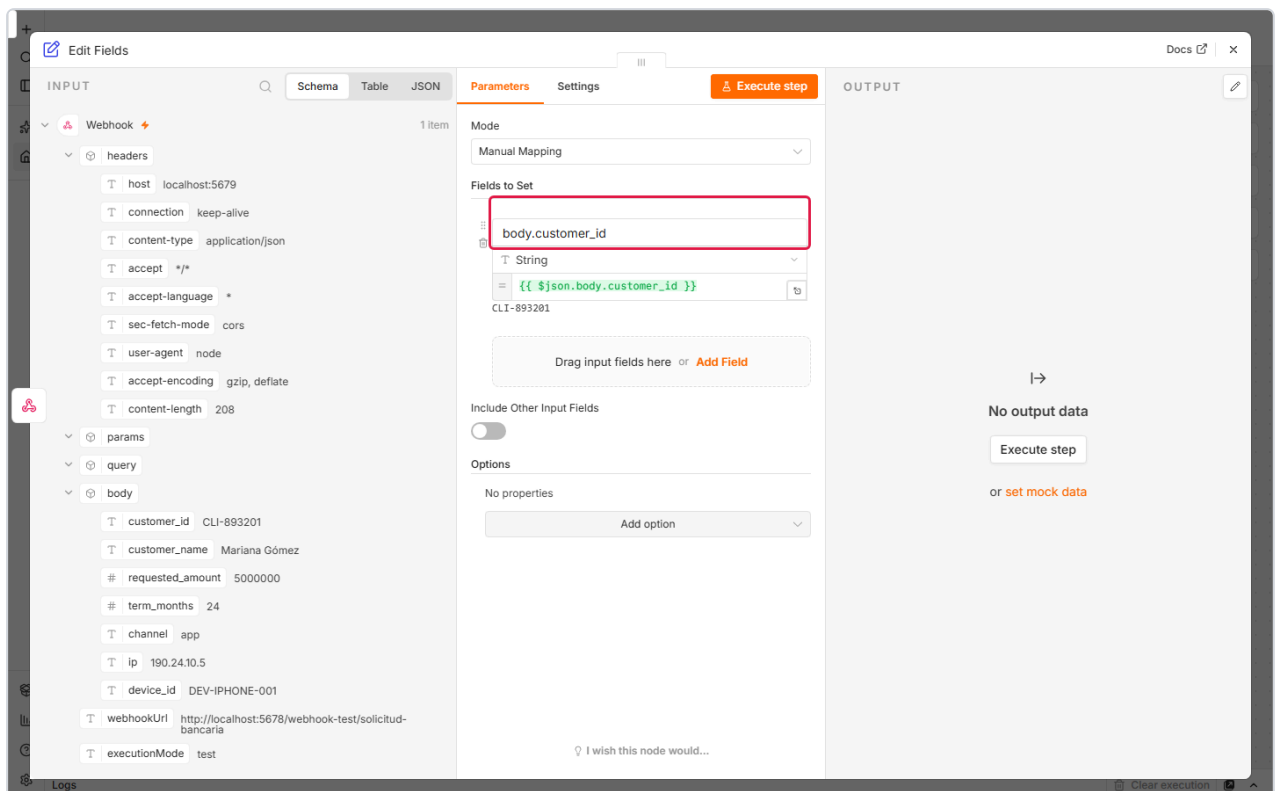
1. Haz clic en el `+` del Webhook, busca `Edit Fields` y elige **Edit Fields (Set)**.



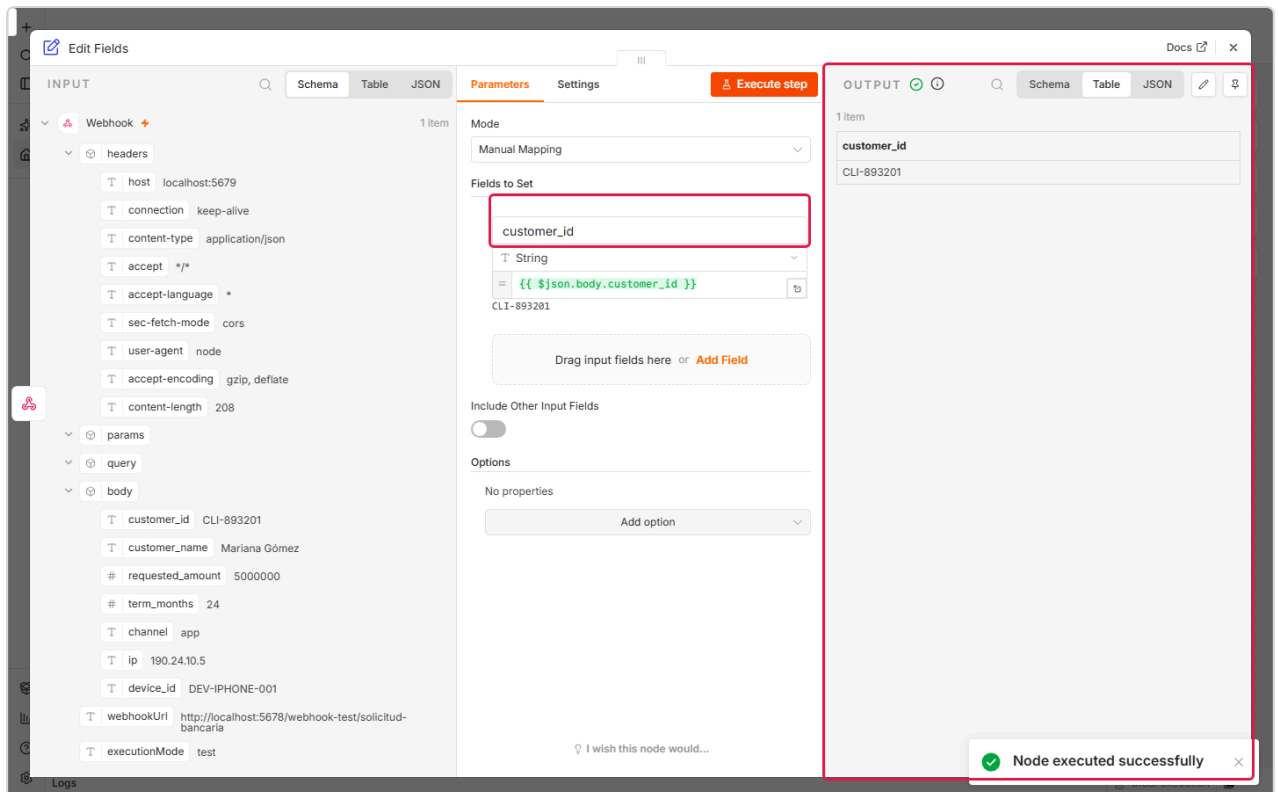
2. Renómbra a `Preparar payload` (clic en el título del panel).
3. Como ya hay datos de prueba, la columna INPUT muestra el `body` recibido. **Arrastra** el campo `customer_id` (dentro de `body`) hasta la zona *Drag input fields here*:



4. n8n crea el campo con el nombre `body.customer_id`. Cámbialo a `customer_id`: los nodos siguientes usan `$json.customer_id`, sin `body`.



Al hacer clic en **Execute step**, OUTPUT muestra el campo con su nombre correcto:



5. Repite con los demás campos hasta tener estos 7 (revisa el **tipo**: los montos son **Number**):

Nombre	Tipo	Valor
customer_id	String	{{ \$json.body.customer_id }}
customer_name	String	{{ \$json.body.customer_name }}
requested_amount	Number	{{ \$json.body.requested_amount }}
term_months	Number	{{ \$json.body.term_months }}
channel	String	{{ \$json.body.channel }}
ip	String	{{ \$json.body.ip }}
device_id	String	{{ \$json.body.device_id }}

The screenshot shows the n8n interface for the 'Preparar payload' node. The 'Parameters' tab is selected, and the 'Fields to Set' section is highlighted with a red box. It shows manual mappings for five fields: customer_id, customer_name, requested_amount, term_months, and channel. Each field is mapped to a JSON path from the body of the incoming message. The 'body' section on the left shows the source data: customer_id (CLI-893201), customer_name (Mariana Gómez), requested_amount (5000000), term_months (24), and channel (app). The 'OUTPUT' tab on the right shows the resulting JSON object with the same fields and values.

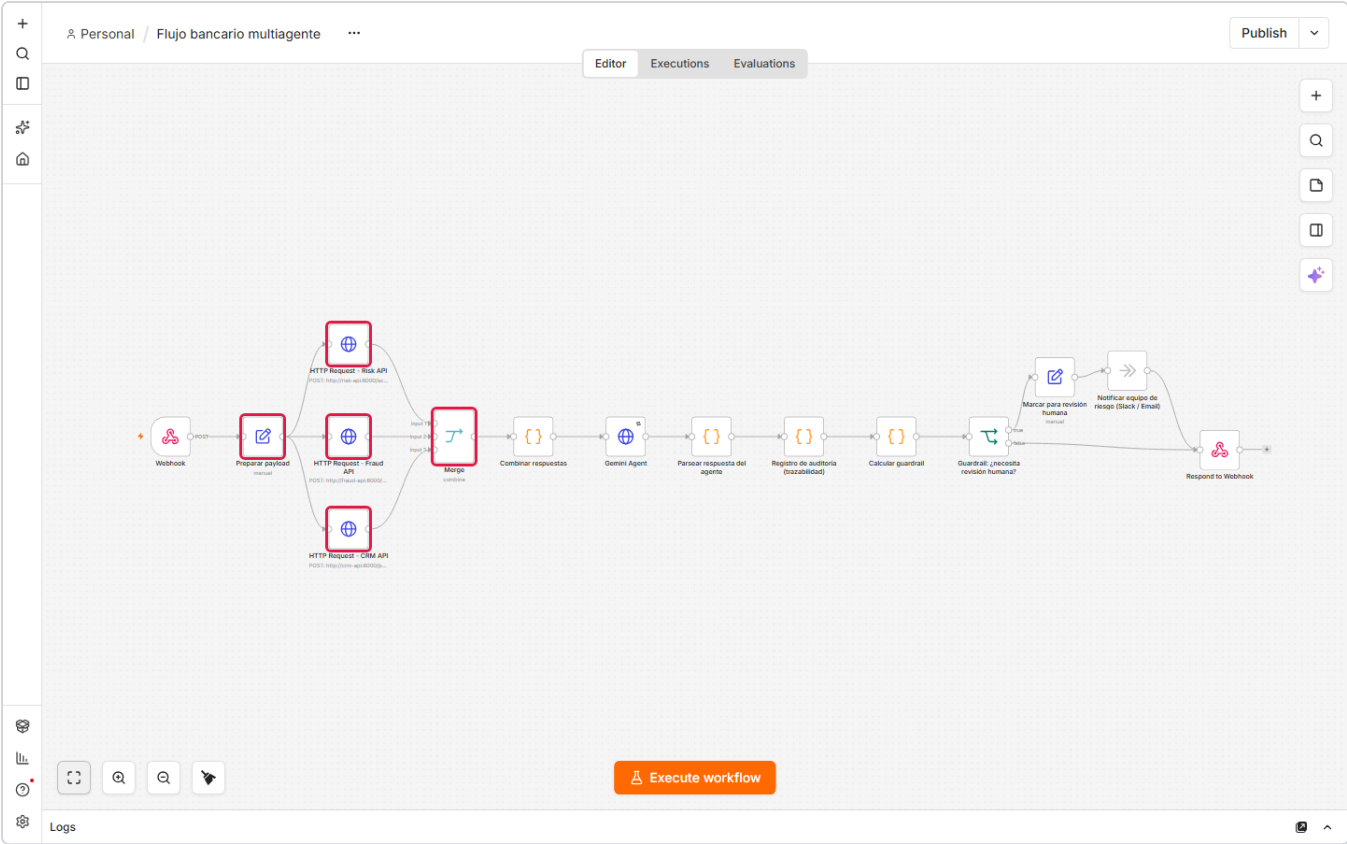
customer_id	customer_name	requested_amount	term_months
CLI-893201	Mariana Gómez	5000000	24

Si prefieres escribir en vez de arrastrar: **Add Field**, escribe el nombre, elige el tipo y en el valor escribe la expresión completa empezando por `{{`. `n8n` cambia el campo a modo *Expression*.

Atajo (como en la solución): en *Edit Fields* cambia **Mode** a **JSON**, pon el campo **JSON** en modo *Expression* y pega el contenido de `snippets/02-preparar-payload.node.json`. Así los 7 valores llegan como texto (también los montos); los servicios los aceptan igual.

6. Pasos 3, 4 y 5 — Risk, Fraud y CRM en paralelo

Los tres servicios se consultan **en paralelo**: los tres salen de *Preparar payload*. **No los encadenes** uno detrás de otro: si lo haces, Fraud y CRM reciben la respuesta de Risk en vez de los datos de la solicitud y fallan.



Cómo crear las tres ramas:

1. Haz clic en el **+** de *Preparar payload*, busca **HTTP Request** y agrégalo. Será **HTTP Request - Risk API**.
2. Para la segunda rama, **arrastra desde el círculo de salida de *Preparar payload*** y suelta en un espacio vacío debajo; busca **HTTP Request**. Será **HTTP Request - Fraud API**.
3. Repite para **HTTP Request - CRM API**.

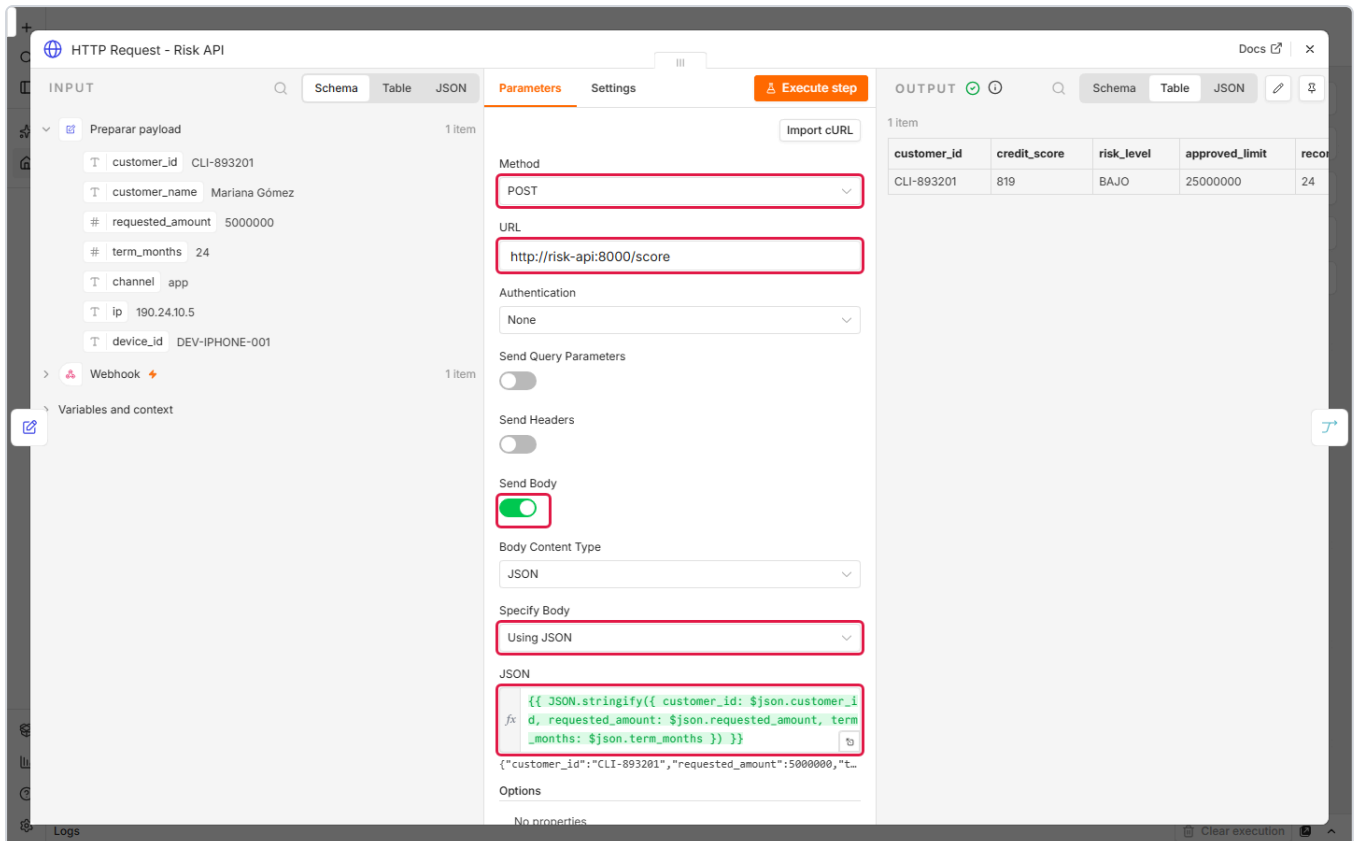
Configura los tres igual, cambiando solo URL y JSON. En todos: **Method** POST, **Authentication** None, activa **Send Body**, **Body Content Type** JSON, **Specify Body** Using JSON y pega el JSON en el campo **JSON** (empieza con `{{`, por eso queda en modo *Expression*).

Nodo	URL
HTTP Request - Risk API	http://risk-api:8000/score
HTTP Request - Fraud API	http://fraud-api:8000/check
HTTP Request - CRM API	http://crm-api:8000/profile

Ojo con la URL: dentro de n8n los servicios se llaman por su nombre (`risk-api`) y puerto interno `8000`. `localhost:8001` solo funciona desde tu navegador, no desde n8n.

JSON de Risk API: — archivo `snippets/03-risk-body.txt` (la línea 1 es la URL; copia solo la última línea, el body):

```
{{ JSON.stringify({ customer_id: $json.customer_id, requested_amount: $json.requested_amount, term_months: $json.term_months }) }}
```



JSON de Fraud API: — archivo `snippets/04-fraud-body.txt` (la línea 1 es la URL; copia solo la última línea, el body):

```
{{ JSON.stringify({ customer_id: $json.customer_id, ip: $json.ip, channel: $json.channel, amount: $json.requested_amount, device_id: $json.device_id }) }}
```

HTTP Request - Fraud API

INPUT

- Preparar payload (1 item)
 - customer_id: CLI-893201
 - customer_name: Mariana Gómez
 - requested_amount: 5000000
 - term_months: 24
 - channel: app
 - ip: 190.24.10.5
 - device_id: DEV-IPHONE-001
- Webhook
- Variables and context

Parameters

- Method: POST
- URL: `http://fraud-api:8000/check`
- Authentication: None
- Send Query Parameters: ☐
- Send Headers: ☐
- Send Body: ☒
- Body Content Type: JSON
- Specify Body: Using JSON
- JSON: `{{ JSON.stringify({ customer_id: $json.customer_id, ip: $json.ip, channel: $json.channel, amount: $json.requested_amount, device_id: $json.device_id }) }}`
- Options: No properties

OUTPUT

customer_id	fraud_risk_score	is_high_risk	alerts
CLI-893201	0	false	suspicious_ip : false high_amount : false risky_channel : false

JSON de CRM API: — archivo `snippets/05-crm-body.txt` (la línea 1 es la URL; copia solo la última línea, el body):

```
{{ JSON.stringify({ customer_id: $json.customer_id }) }}
```

HTTP Request - CRM API

INPUT

- Preparar payload (1 item)
 - customer_id: CLI-893201
 - customer_name: Mariana Gómez
 - requested_amount: 5000000
 - term_months: 24
 - channel: app
 - ip: 190.24.10.5
 - device_id: DEV-IPHONE-001
- Webhook
- Variables and context

Parameters

- Method: POST
- URL: `http://crm-api:8000/profile`
- Authentication: None
- Send Query Parameters: ☐
- Send Headers: ☐
- Send Body: ☒
- Body Content Type: JSON
- Specify Body: Using JSON
- JSON: `{{ JSON.stringify({ customer_id: $json.customer_id }) }}`
- Options: No properties

OUTPUT

customer_id	customer_name	segment	monthly_income
CLI-893201	Mariana Gómez	PREFERENCIAL	6200000

Con cada nodo puedes hacer clic en **Execute step** y ver la respuesta del servicio en OUTPUT.

7. Paso 6 — Merge

Junta las tres respuestas en un solo ítem.

1. Haz clic en el **+** de **HTTP Request - Risk API**, busca **Merge** y agrégalo. Queda conectado a la entrada **Input 1**.
2. Configura:

Campo	Valor
Mode	Combine
Combine By	Position
Number of Inputs	3

3. Ahora el Merge muestra tres entradas. Arrastra la salida de **Fraud API** a **Input 2** y la de **CRM API** a **Input 3** (el orden importa: Risk, Fraud, CRM).

The screenshot shows the n8n workflow editor with a 'Merge' node configured. The left sidebar lists the inputs: 'HTTP Request - Risk API (Input 1)', 'HTTP Request - Fraud API (Input 2)', and 'HTTP Request - CRM API (Input 3)'. The 'Parameters' tab is active, showing 'Mode' set to 'Combine', 'Combine By' set to 'Position', and 'Number of Inputs' set to '3'. The 'OUTPUT' panel on the right displays a single item with a table of data: customer_id (CLI-893201), credit_score (819), risk_level (BAJO), approved_limit (25000000), and recommended_term_months (24). The bottom status bar indicates 'Clear execution'.

8. Paso 7 — Combinar respuestas

Reorganiza el objeto plano del Merge en tres bloques: `cliente`, `riesgo` y `fraude`.

1. En el `+` del Merge busca `Code` y elige **Code** (si pregunta, *Code in JavaScript*).
2. **Renómbra**lo exactamente `Combinar respuestas`. Los pasos 10 y 11 leen este nodo con `$( 'Combinar respuestas' )`; si se llama distinto (por ejemplo *Code*), esos pasos fallan.
3. Deja **Mode** `Run Once for All Items` y **Language** `JavaScript`. Borra el código de ejemplo (`Ctrl+A`, `Supr`) y pega el contenido de `snippets/07-combinar-respuestas.js`:

```
// El Merge de arriba junta Risk + Fraud + CRM en un solo objeto plano.
// Aqui lo reordenamos en la misma forma anidada que usa el resto del flujo.
const flat = $json;

return [
  {
    json: {
      cliente: {
        customer_id: flat.customer_id,
        customer_name: flat.customer_name,
        segment: flat.segment,
        monthly_income: flat.monthly_income,
        employment: flat.employment,
        tenure_months: flat.tenure_months,
        product_history: flat.product_history,
      },
      riesgo: {
        customer_id: flat.customer_id,
        credit_score: flat.credit_score,
        risk_level: flat.risk_level,
        approved_limit: flat.approved_limit,
        recommended_term_months: flat.recommended_term_months,
      },
      fraude: {
        customer_id: flat.customer_id,
        fraud_risk_score: flat.fraud_risk_score,
        is_high_risk: flat.is_high_risk,
        alerts: flat.alerts,
      },
    },
  },
];
```

Combinar respuestas

INPUT

Schema Table JSON

Parameters Settings

Execute step

Mode

Run Once for All Items

Language

JavaScript

JavaScript

```
1 // El Merge de arriba junta Risk + Fraud + CRM en un solo objeto plano.
2 // Aquí lo reordenamos en la misma forma anidada que usa el resto del
  flujo.
3 const flat = $json;
4
5 return [
6   {
7     json: {
8       cliente: {
9         customer_id: flat.customer_id,
10        customer_name: flat.customer_name,
11        segment: flat.segment,
12        monthly_income: flat.monthly_income,
13        employment: flat.employment,
14        tenure_months: flat.tenure_months,
15        product_history: flat.product_history,
16      },
17      riesgo: {
```

Type \$ for a list of special vars/methods. Debug by using console.log() statements and viewing their output in the browser console.

OUTPUT

Schema Table JSON

1 item

cliente	riesgo
customer_id : CLI-893201	customer_id : CLI-8
customer_name : Mariana Gómez	credit_score : 819
segment : PREFERENCIAL	risk_level : BAJO
monthly_income : 6200000	approved_limit : 25
employment : Empleado	recommended_term
tenure_months : 42	
product_history	
0 : tarjeta	
1 : cuenta de ahorros	

Al ejecutarlo, OUTPUT muestra las columnas cliente, riesgo y fraude.

9. Paso 8 — Gemini Agent (el modelo)

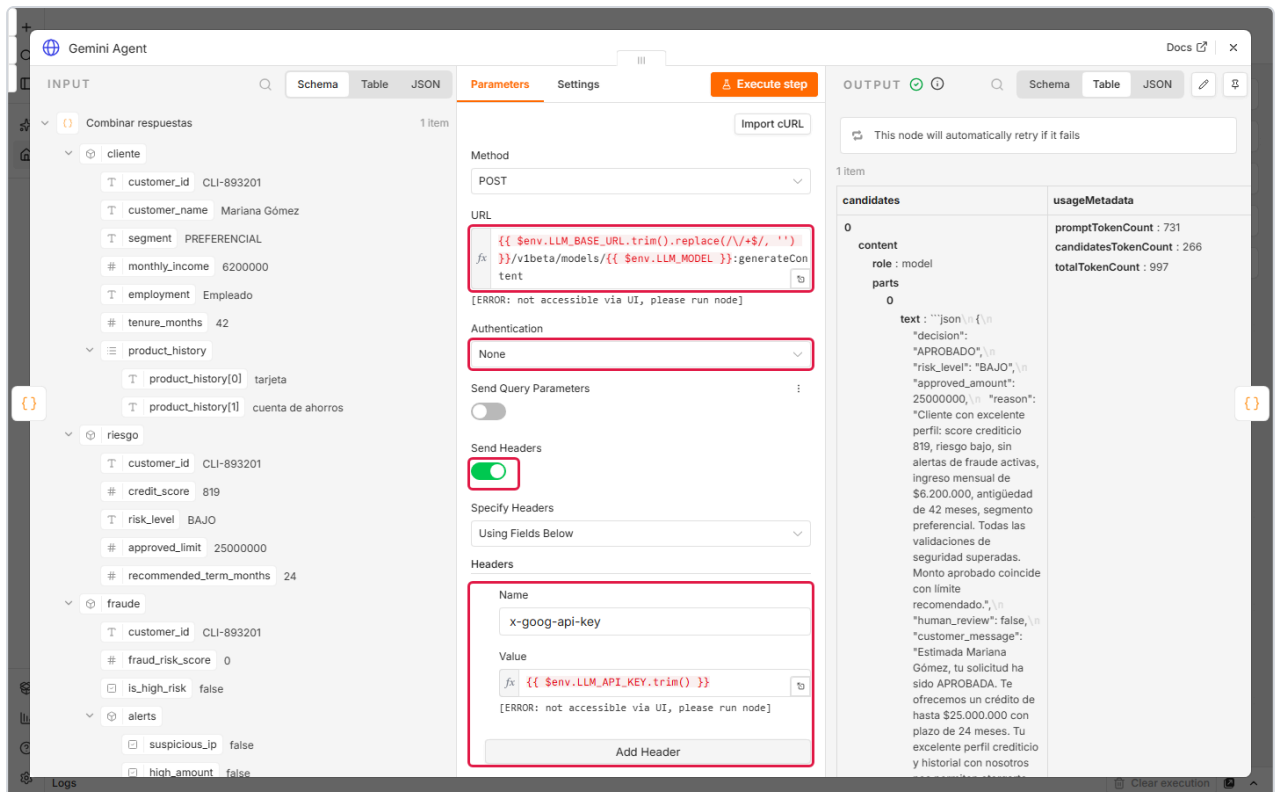
Este es el **único** nodo que llama al modelo. Es un nodo **HTTP Request** normal (no el nodo *AI Agent* de n8n).

1. En el **+** de *Combinar respuestas* busca **HTTP Request** y renómbralo **Gemini Agent**.
2. Configura (los valores con `{{ }}` van en modo *Expression*):

Campo	Valor
Method	POST
URL	<code>{{ \$env.LLM_BASE_URL.trim().replace(/\\$/ , '') }}v1beta/models/{{ \$env.LLM_MODEL }}:generateContent (línea 1 de 08-gemini-agent.txt)</code>
Authentication	None
Send Headers	activado
Specify Headers	Using Fields Below
Header → Name	x-goog-api-key
Header → Value	<code>{{ \$env.LLM_API_KEY.trim() }}</code>
Send Body	activado
Body Content Type	JSON
Specify Body	Using JSON

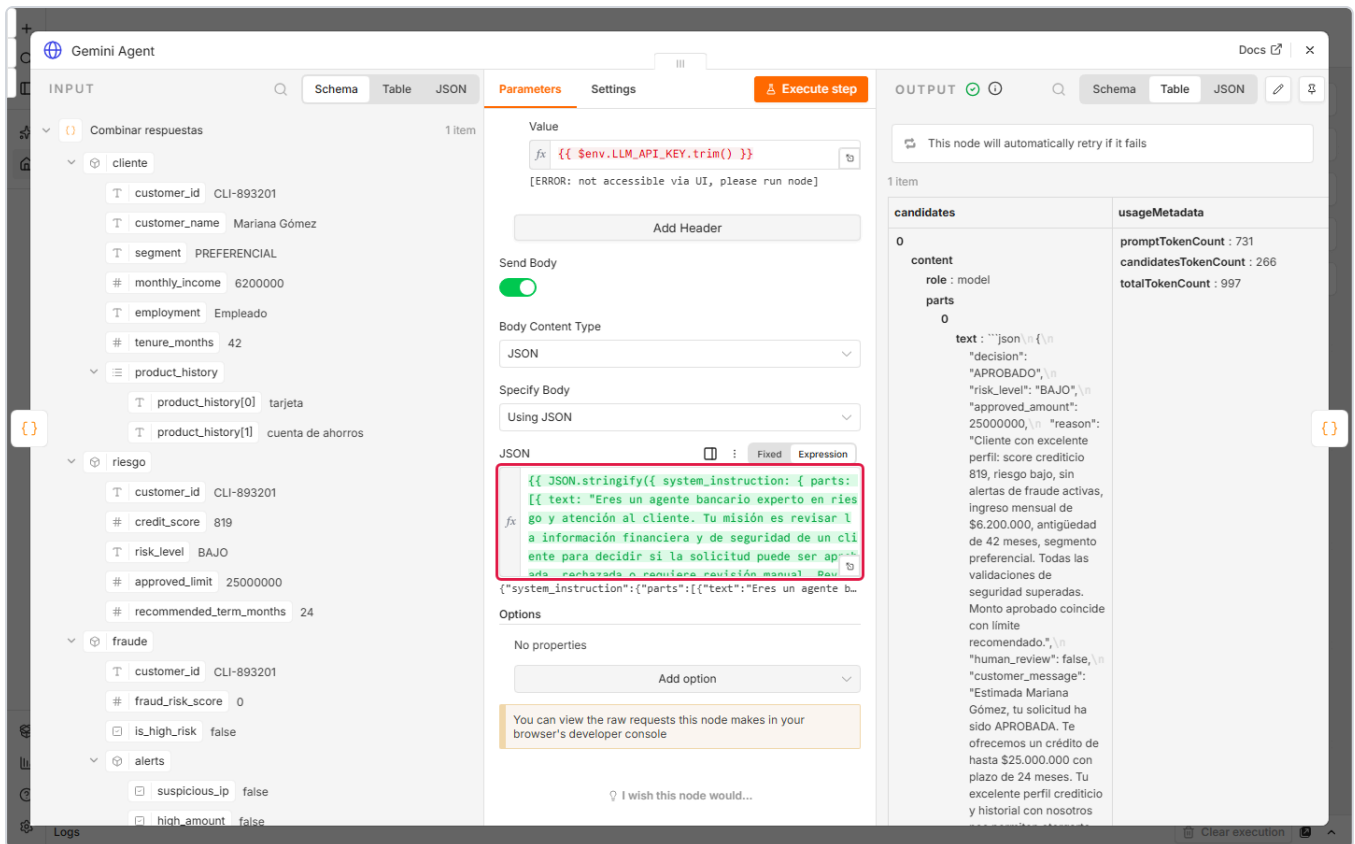
La URL y la llave salen de tu `.env` (`LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`), así que el mismo flujo funciona con la llave del taller o con la tuya de Gemini. **No se crea ninguna credencial.**

Debajo de la URL y del valor del header verás `[ERROR: not accessible via UI, please run node]`. **Es normal:** el editor no muestra variables de entorno en la vista previa; al ejecutar sí se resuelven.



- En **JSON** pega el cuerpo completo: la última línea de **snippets/08-gemini-agent.txt** (ese archivo trae, separados por una línea vacía: URL, nombre del header, valor del header y body). Es una sola línea larga; cópiala entera:

```
{{ JSON.stringify({ system_instruction: { parts: [{ text: "Eres un agente bancario experto en riesgo y a
tención al cliente. Tu misión es revisar la información financiera y de seguridad de un cliente para dec
idir si la solicitud puede ser aprobada, rechazada o requiere revisión manual. Revisa los datos del clie
nte, el score de riesgo, la evaluación de fraude y la capacidad de endeudamiento. El campo decision DEBE
ser exactamente uno de estos cuatro valores, sin variaciones: APROBADO, APROBADO_CON_RESTRICCIONES, RECH
AZADO, PENDIENTE_REVISION_HUMANA. Usa APROBADO solo cuando no exista ninguna alerta de fraude activa (al
erts.suspicious_ip, alerts.high_amount, alerts.risky_channel todas en false) y el monto aprobado sea igu
al al límite completo. Usa APROBADO_CON_RESTRICCIONES cuando el riesgo sea bajo o medio pero exista al m
enos una alerta de fraude activa, o cuando el monto aprobado sea menor al solicitado. Usa RECHAZADO cuan
do el riesgo sea alto y no proceda ninguna aprobación. Usa PENDIENTE_REVISION_HUMANA solo si tú mismo co
nsideras que el caso es ambiguo y requiere que un humano decida. Devuelve un resultado con: decisión, ra
zón, monto sugerido, riesgo y mensaje final para el cliente. Hazlo con lenguaje claro, conservador y ori
entado a la seguridad. Responde ÚNICAMENTE con un objeto JSON válido (sin texto adicional, sin markdown)
con las claves: decision, risk_level, approved_amount, reason, human_review, customer_message." } } }, co
ntents: [ { parts: [ { text: JSON.stringify($json) } ] } ] }) }}
```



- Recomendado:** en la pestaña **Settings** del nodo activa **Retry On Fail** con **Max Tries** 3 y **Wait Between Tries (ms)** 2000. Si el modelo responde con un error pasajero (por ejemplo, el límite por minuto), n8n reintenta solo.

Al ejecutar, OUTPUT muestra `candidates` → `content` → `parts` → `text`: ahí viene la decisión del modelo como texto JSON (a veces envuelta en ```json`). El paso siguiente la convierte en datos.

10. Pasos 9, 10 y 11 — Parsear, auditar y calcular el guardrail

Son tres nodos **Code** seguidos. Para cada uno: `+` → `Code`, renómbralo, deja *Run Once for All Items* / *JavaScript*, borra el ejemplo y pega el código **desde el archivo de snippets/** que indica cada paso (no desde el PDF: los saltos de línea rompen el código).

Paso 9 — Parsear respuesta del agente

Archivo: `snippets/09-parsear-respuesta.js`

Saca el texto de la respuesta del modelo y lo convierte en JSON.

```
// Gemini devuelve { candidates: [ { content: { parts: [ { text: '...' } ] } } ], ... }
// El texto es el JSON que le pedimos al agente en el prompt.
let raw = $json.candidates[0].content.parts[0].text.trim();

// Por si el modelo lo envuelve en un bloque de código markdown
if (raw.startsWith('```')) {
  raw = raw.replace(/^```(json)?/, '').replace(/```$/, '').trim();
}

const parsed = JSON.parse(raw);
return [{ json: parsed }];
```

The screenshot shows the n8n workflow editor for a node titled "Parsear respuesta del agente".

- INPUT:** A tree view showing the data structure. It includes a "Gemini Agent" node with a "candidates" array. The "candidates[0]" object has a "content" object with a "parts" array. The "parts[0]" object has a "text" field containing a JSON string.
- Editor:** The JavaScript code from the snippet is pasted into the editor. It uses `$json` to access the input data, trims the text, removes markdown backticks if present, and then uses `JSON.parse` to parse the string into a JavaScript object. The result is returned as an array of objects with a `json` property.
- OUTPUT:** A table showing the output of the node. It has four columns: `decision`, `risk_level`, `approved_amount`, and `reason`. The output is an array of objects, with the first object shown as: `{ decision: 'APROBADO', risk_level: 'BAJO', approved_amount: '25000000', reason: '...' }`.

Paso 10 — Registro de auditoría (trazabilidad)

Archivo: `snippets/10-auditoria.js`

Arma el registro de auditoría (lo imprime en el log) y deja pasar la decisión sin cambios. Lee los nodos **Combinar respuestas** y **Gemini Agent** **por nombre**.

```
// Paso 7 - Trazabilidad y auditoría.
// Arma el registro completo de la decisión. Reemplaza el console.log
// de abajo por un nodo real (Postgres, Elasticsearch, etc.) cuando
// tengas esa base de datos lista; mientras tanto, cada ejecución queda
// visible en n8n -> Executions con este objeto en el log.
const combinado = $('Combinar respuestas').item.json;
const decision = $json;

const audit = {
  request_id: `${$workflow.id}-${Date.now()}`,
  timestamp: new Date().toISOString(),
  customer_id: combinado.cliente.customer_id,
  risk_response: combinado.riesgo,
  fraud_response: combinado.fraude,
  crm_response: combinado.cliente,
  // modelo que realmente respondió (Gemini o el gateway del taller lo informan en modelVersion)
  llm_model: $('Gemini Agent').item.json.modelVersion ?? $env.LLM_MODEL,
  decision: decision.decision,
  human_review: decision.human_review,
};

// TODO: inserta 'audit' en tu tabla/índice de trazabilidad real.
console.log('AUDITORIA:', JSON.stringify(audit));

return [{ json: decision }];
```

Registro de auditoría (trazabilidad)

INPUT

- Parsear respuesta del agente (1 item)
 - decision: APROBADO
 - risk_level: BAJO
 - approved_amount: 25000000
 - reason: Cliente con excelente perfil: score crediticio 819, riesgo bajo, sin alertas de fraude activas, Ingreso mensual de \$6.200.000, antigüedad de 42 meses, segmento preferencial. Todas las validaciones de seguridad superadas. Monto aprobado coincide con límite recomendado.
 - human_review: false
 - customer_message: Estimada Mariana Gómez, tu solicitud ha sido APROBADA. Te ofrecemos un crédito de hasta \$25.000.000 con plazo de 24 meses. Tu excelente perfil crediticio y historial con nosotros nos permiten otorgarte esta aprobación. Dirígete a cualquiera de nuestras sucursales o plataforma digital para formalizar la solicitud. ¡Gracias por tu confianza!

Parameters

Mode: Run Once for All Items

Language: JavaScript

JavaScript

```
1 // Paso 7 - Trazabilidad y auditoría.
2 // Arma el registro completo de la decisión. Reemplaza el console.log
3 // de abajo por un nodo real (Postgres, Elasticsearch, etc.) cuando
4 // tengas esa base de datos lista; mientras tanto, cada ejecución queda
5 // visible en n8n -> Executions con este objeto en el log.
6 const combinado = $('Combinar respuestas').item.json;
7 const decision = $json;
8
9 const audit = {
10   request_id: `${$workflow.id}-${Date.now()}`,
11   timestamp: new Date().toISOString(),
12   customer_id: combinado.cliente.customer_id,
13   risk_response: combinado.riesgo,
14   fraud_response: combinado.fraude,
15   crm_response: combinado.cliente,
16   // modelo que realmente respondió (Gemini o el gateway del taller lo
17   // informan en modelVersion)
18   llm_model: $('Gemini Agent').item.json.modelVersion ?? $env.LLM_MODEL,
19   decision: decision.decision,
20   human_review: decision.human_review,
21 };
22
23 // TODO: inserta 'audit' en tu tabla/índice de trazabilidad real.
24 console.log('AUDITORIA:', JSON.stringify(audit));
25
26 return [{ json: decision }];
```

OUTPUT

decision	risk_level	approved_amount	reason
APROBADO	BAJO	25000000	Cliente con excelente perfil: score crediticio 819, riesgo bajo, sin alertas de fraude activas, Ingreso mensual de \$6.200.000, antigüedad de 42 meses, segmento preferencial. Todas las validaciones de seguridad superadas. Monto aprobado coincide con límite recomendado.

Paso 11 — Calcular guardrail

Archivo: `snippets/11-guardrail.js`

Regla de negocio que **no depende del modelo**: fuerza revisión humana si hay fraude alto, o si el monto es alto y el riesgo es ALTO. Agrega el campo `needs_human_review`.

```
// Guardrail combinado: un monto alto por si solo no basta para forzar
// revision humana si el riesgo es bajo; solo cuenta cuando coincide con
// riesgo ALTO. Un fraude alto siempre fuerza revision, sin importar el monto.
const decision = $json;
const fraude = $('Combinar respuestas').item.json.fraude;

const montoAltoYRiesgoAlto = decision.approved_amount > 10000000 && decision.risk_level === 'ALTO';
const fraudeAlto = fraude.fraud_risk_score >= 45;

const needs_human_review = montoAltoYRiesgoAlto || fraudeAlto;

return [{ json: { ...decision, needs_human_review } }];
```

The screenshot shows the n8n workflow editor for a step named 'Calcular guardrail'. The left sidebar lists the workflow steps: 'Registro de auditoría (trazabilidad)', 'Parsear respuesta del agente', 'Gemini Agent', 'Combinar respuestas', 'Merge', 'HTTP Request - Risk API', 'HTTP Request - Fraud API', 'HTTP Request - CRM API', and 'Preparar payload'. The main editor area has tabs for 'Parameters', 'Settings', and 'JavaScript'. The 'Parameters' tab is active, showing a 'Mode' dropdown set to 'Run Once for All Items' and a 'Language' dropdown set to 'JavaScript'. The 'JavaScript' tab is also active, displaying the code from the previous block. The 'OUTPUT' tab on the right shows a table with 4 columns: 'decision', 'risk_level', 'approved_amount', and 'reason'. The first row contains the values 'APROBADO', 'BAJO', '25000000', and a detailed text description of the customer's credit profile and the approval decision.

decision	risk_level	approved_amount	reason
APROBADO	BAJO	25000000	Cliente con excelente perfil: score crediticio 819, riesgo bajo, sin alertas de fraude activas, ingreso mensual de \$6.200.000, antigüedad de 42 meses, segmento preferencial. Todas las validaciones de seguridad superadas. Monto aprobado coincide con límite recomendado.

11. Paso 12 — If: ¿necesita revisión humana?

1. En el + de *Calcular guardrail* busca **If** y renómbralo **Guardrail: ¿necesita revisión humana?**.
2. En **Conditions**:
 - En el primer campo arrastra **needs_human_review** desde INPUT (o, en modo *Expression*, pega `{{ $json.needs_human_review }}` desde [snippets/12-if-body.txt](#)).
 - En el operador elige **Boolean** → **is true**.

The screenshot shows the n8n workflow editor for a node named "Guardrail: ¿necesita revisión humana?". The node is configured with the following parameters:

- Inputs:**
 - `decision`: APROBADO
 - `risk_level`: BAJO
 - `approved_amount`: 25000000
 - `reason`: Cliente con excelente perfil: score crediticio 819, riesgo bajo, sin alertas de fraude activas, ingreso mensual de \$6.200.000, antigüedad de 42 meses, segmento preferencial. Todas las validaciones de seguridad superadas. Monto aprobado coincide con límite recomendado.
 - `human_review`: false
 - `customer_message`: Estimada Mariana Gómez, tu solicitud ha sido APROBADA. Te ofrecemos un crédito de hasta \$25.000.000 con plazo de 24 meses. Tu excelente perfil crediticio y historial con nosotros nos permiten otorgarte esta aprobación. Dirígete a cualquiera de nuestras sucursales o plataforma digital para formalizar la solicitud. ¡Gracias por tu confianza!
 - `needs_human_review`: false
- Conditions:**
 - Expression: `{{ $json.needs_human_review }}`
 - Operator: **is true**
- Options:**
 - Convert types where required: ☐
 - No properties

The **OUTPUT** section shows the results of the condition:

True Branch				False Branch (1 item)			
decision	risk_level	approved_amount	reason	decision	risk_level	approved_amount	reason
APROBADO	BAJO	25000000	Client...				

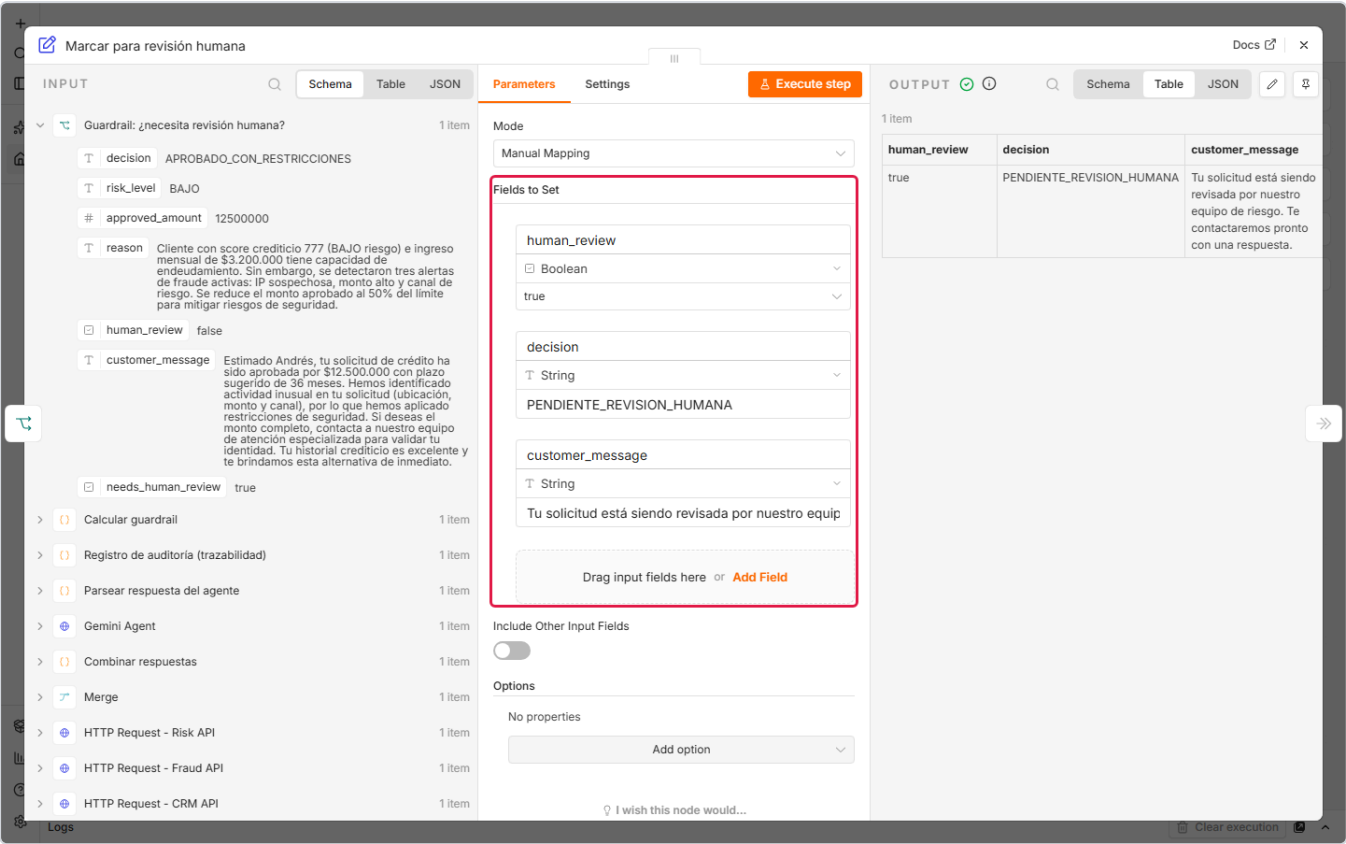
El If tiene **dos salidas**: **true** (necesita revisión) y **false** (respuesta directa). En OUTPUT puedes ver por cuál salió el ítem en las pestañas *True Branch* / *False Branch*.

12. Pasos 13 y 14 — Rama de revisión humana

Paso 13 — Marcar para revisión humana

- Desde la salida **true** del If, + → **Edit Fields (Set)**, renómbalo **Marcar para revisión humana**.
- Agrega estos tres campos (valores fijos, no expresiones):

Nombre	Tipo	Valor
human_review	Boolean	true
decision	String	PENDIENTE_REVISION_HUMANA
customer_message (archivo 13-mensaje-revision.txt)	String	Tu solicitud está siendo revisada por nuestro equipo de riesgo. Te contactaremos pronto con una respuesta.



Paso 14 — Notificar equipo de riesgo (Slack / Email)

El PDF propone un nodo *Send Email*, pero **así no funciona en el taller**: usa `customer_email`, que la solicitud no trae, necesita una credencial de correo (SMTP) que no se configura, y su salida reemplazaría la decisión que debe devolver el webhook. En su lugar usa un marcador de posición:

- En el + de *Marcar para revisión humana* busca **No Operation** y elige **No Operation, do nothing**.
- Renómbalo **Notificar equipo de riesgo (Slack / Email)**.

No se configura nada: deja pasar los datos tal cual. En un sistema real aquí iría la notificación.

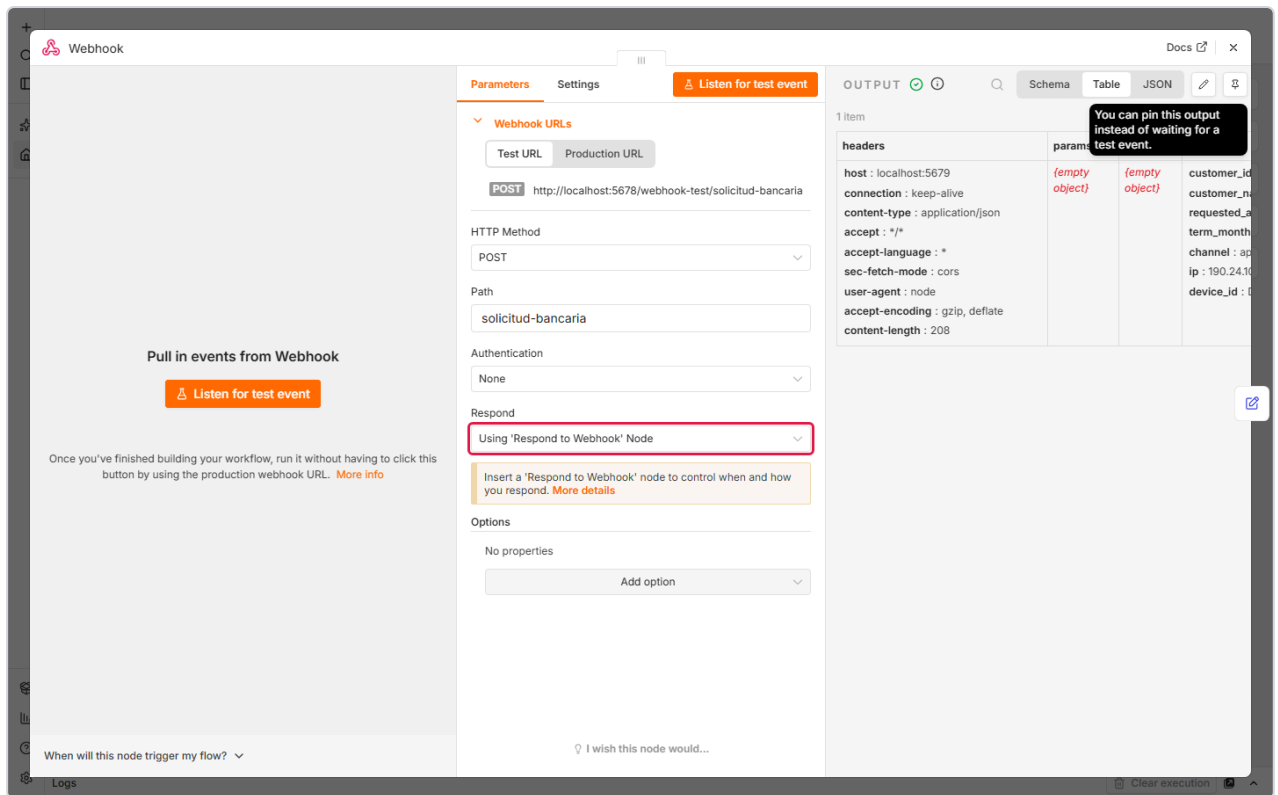
13. Paso 15 — Respond to Webhook

Devuelve la respuesta final a quien envió la solicitud, venga de cualquiera de las dos ramas.

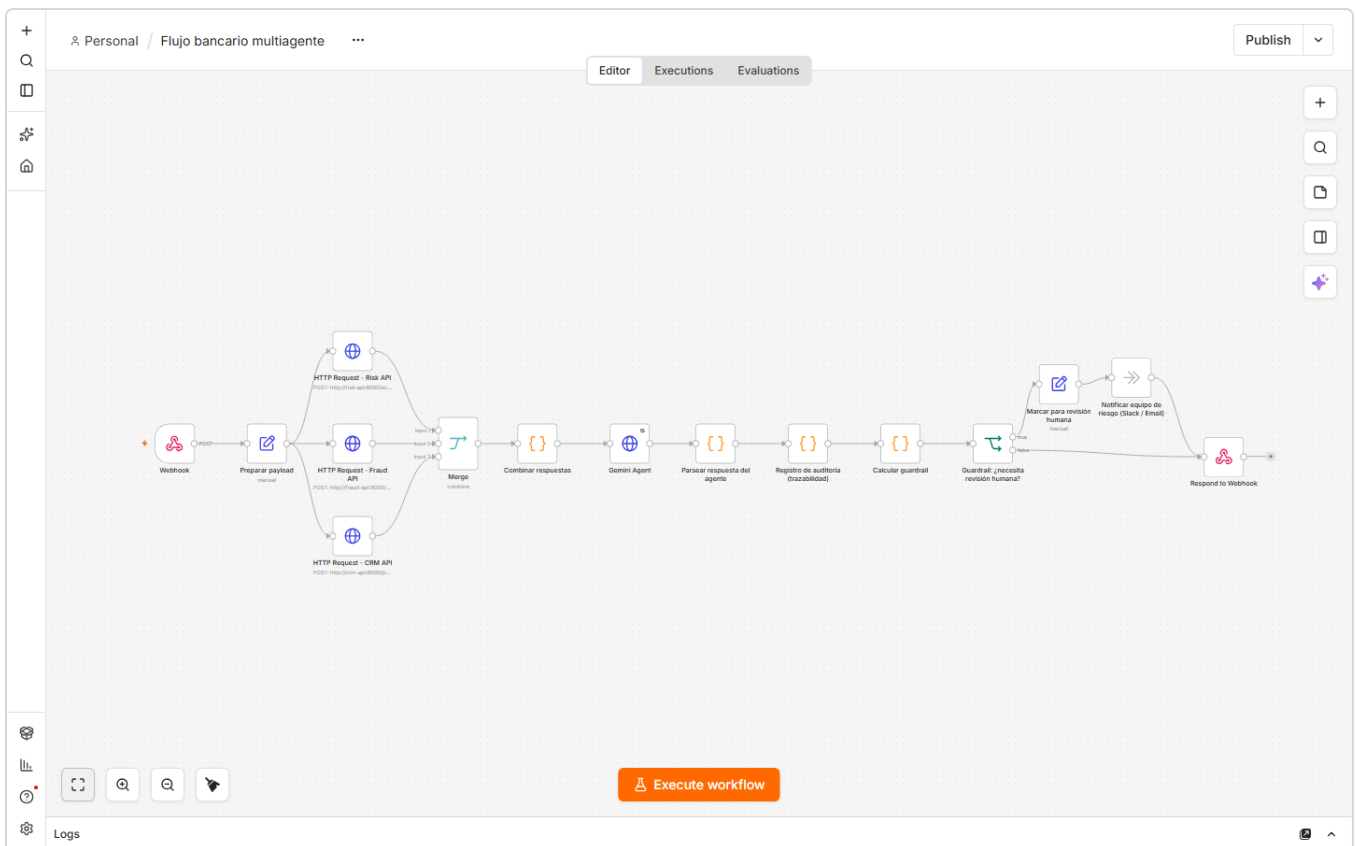
- 1. En el **+** del nodo *Notificar equipo de riesgo* busca **Respond to Webhook** y agrégalo.
- 2. Conecta también la salida **false** del If a este mismo nodo (arrastra desde el círculo *false* hasta la entrada de *Respond to Webhook*).
- 3. Configura:

Campo	Valor
Respond With	JSON
Response Body	{{ \$json }} (modo <i>Expression</i> ; archivo 14-respond-webhook.txt)

- 4. **Vuelve al nodo Webhook** y cambia **Respond** a **Using 'Respond to Webhook' Node**. Ahora sí existe ese nodo.

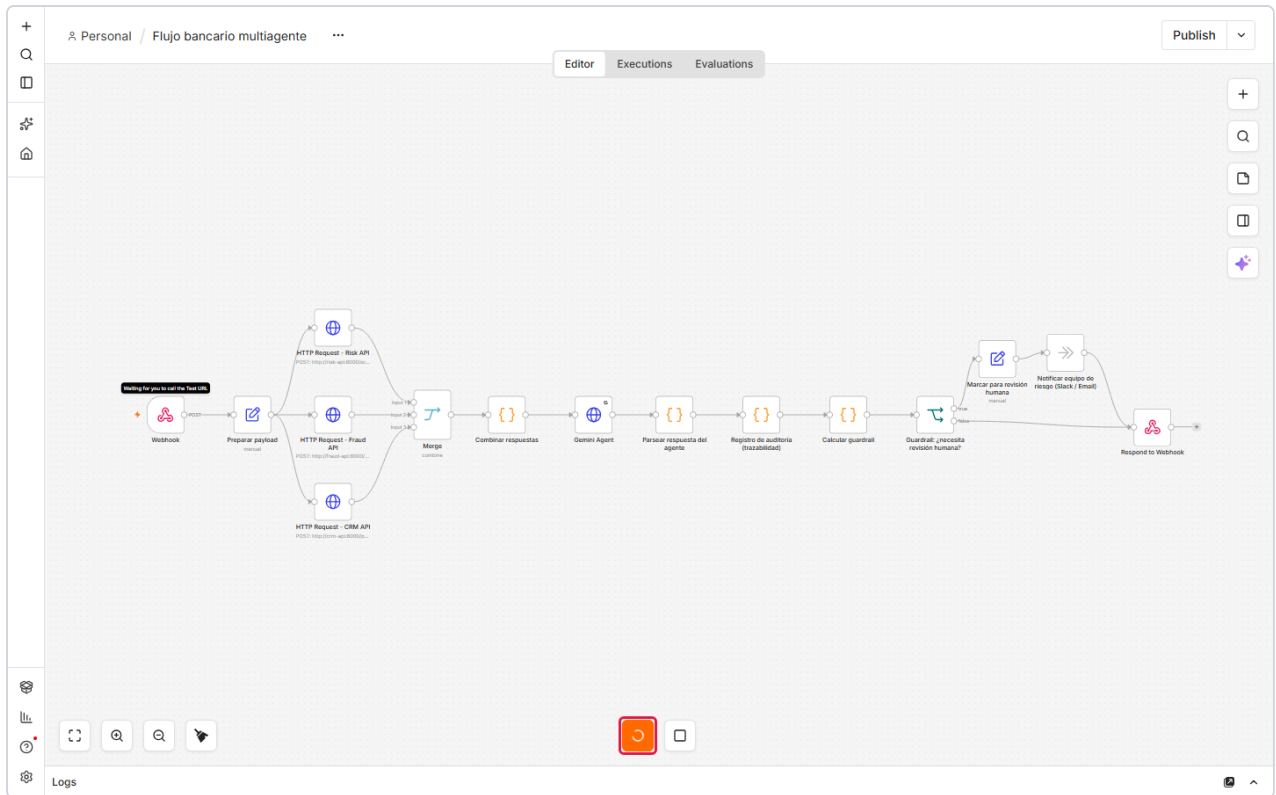


El flujo completo queda así:



14. Probar el flujo completo

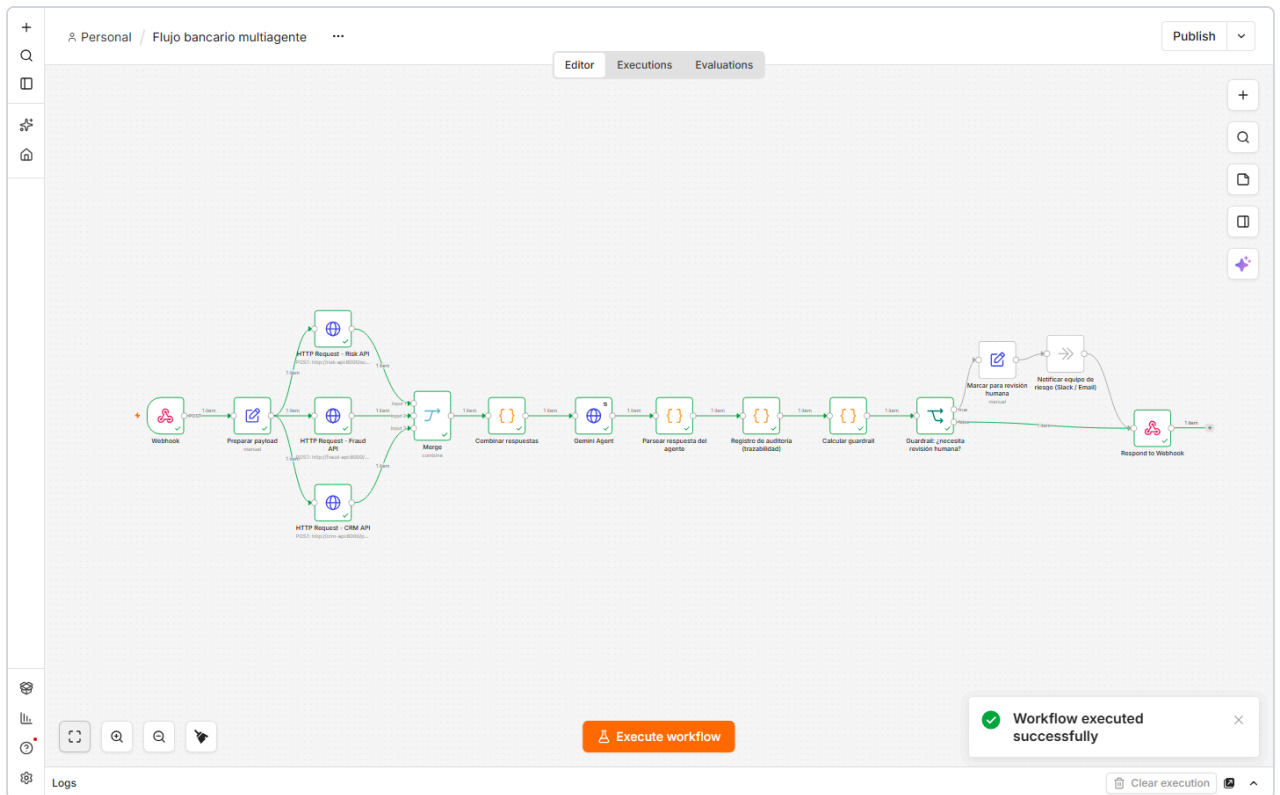
1. Si fijaste (pin) los datos del Webhook, quítales el pin (abre el Webhook y haz clic otra vez en ).
2. Haz clic en **Execute workflow** (botón naranja de abajo). Queda esperando la solicitud:



3. Desde la terminal envía el caso 1:

```
.\scripts\test-flow.ps1 -Test -Files requests\01-cliente-preferencial.json
```

Todos los nodos quedan en verde con *1 item*, la rama **false** del If va directo a la respuesta, y la terminal muestra la decisión del modelo (por ejemplo `"decision": "APROBADO"`).

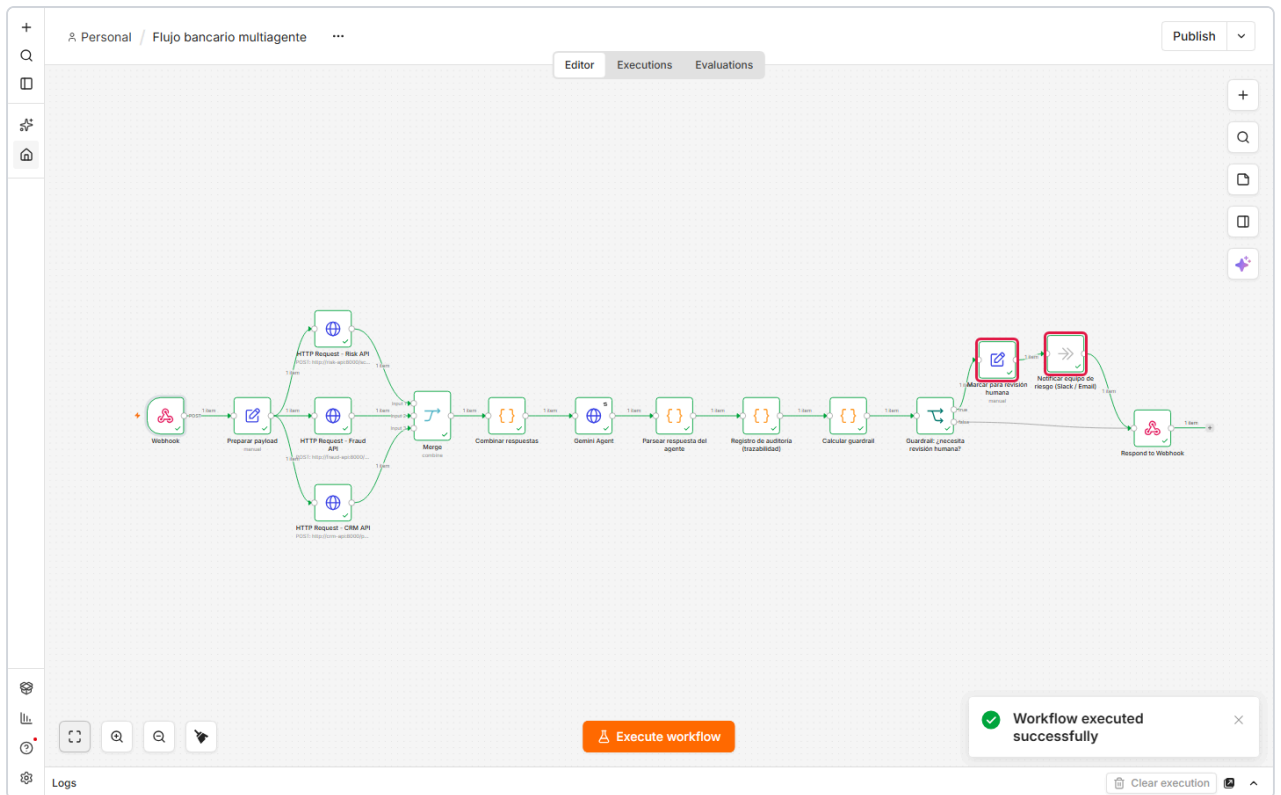


4. Repite (clic en **Execute workflow** y enviar) con el caso de **fraude alto**:

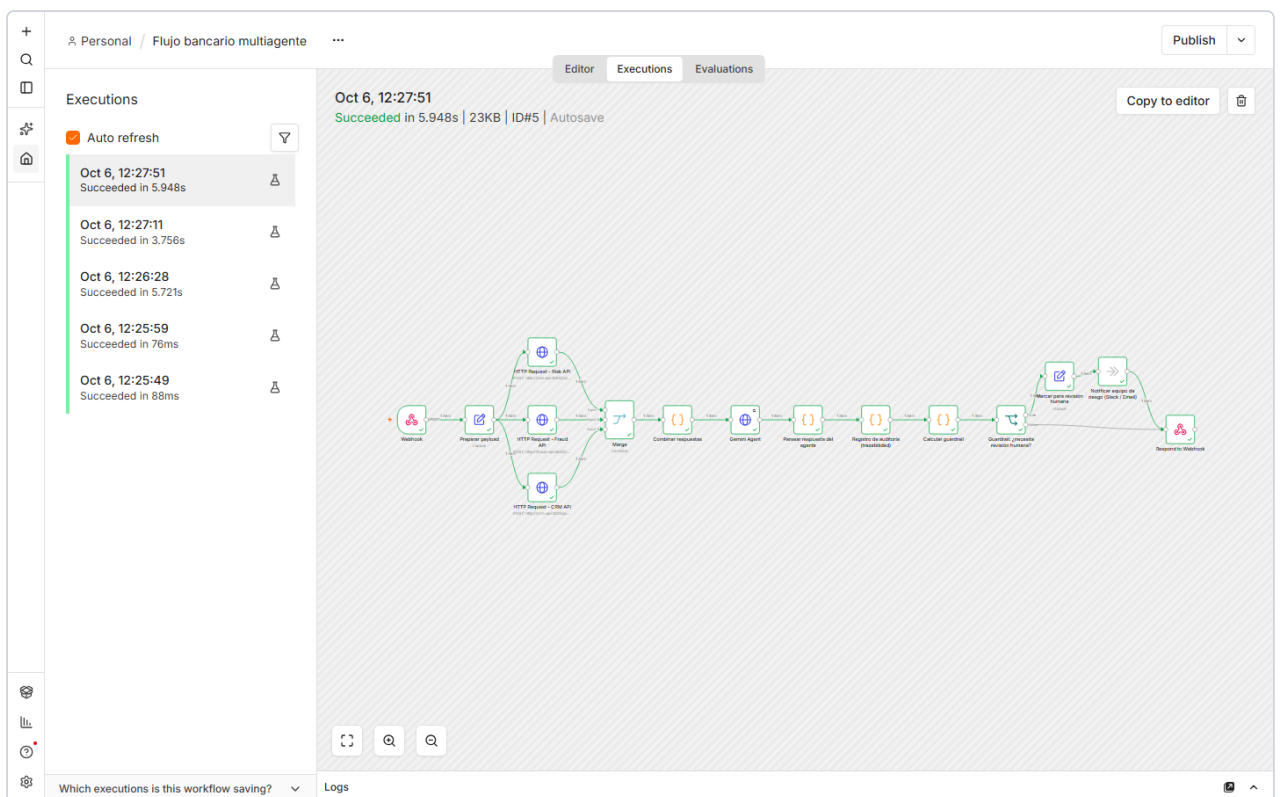
```
.\scripts\test-flow.ps1 -Test -Files requests\02-fraude-alto.json
```

Ahora el ítem sale por la rama **true** y la respuesta es:

```
{
  "human_review": true,
  "decision": "PENDIENTE_REVISION_HUMANA",
  "customer_message": "Tu solicitud está siendo revisada por nuestro equipo de riesgo. Te contactaremos pronto con una respuesta."
}
```



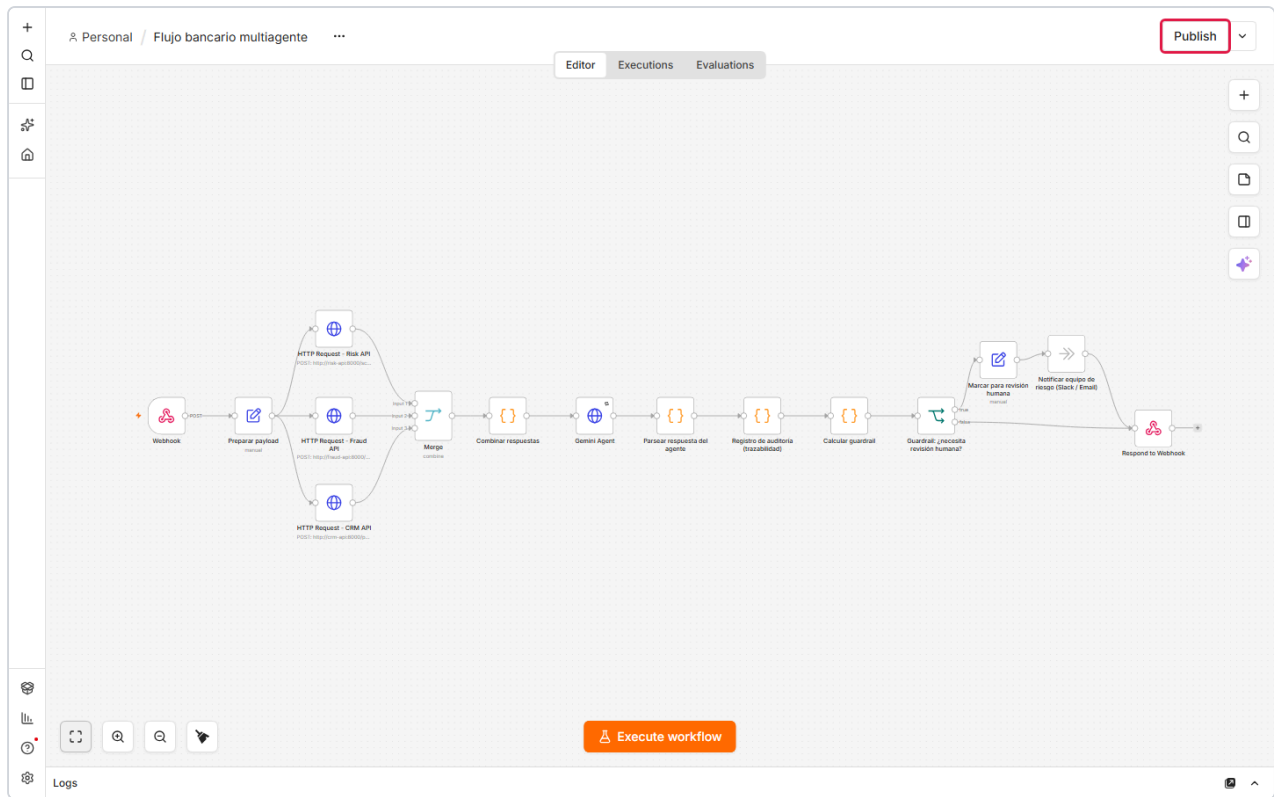
5. En la pestaña **Executions** (arriba al centro) queda el historial de ejecuciones. Al abrir una ves los datos de cada nodo; el registro de auditoría del paso 10 se imprime en el log del contenedor (docker compose logs n8n).



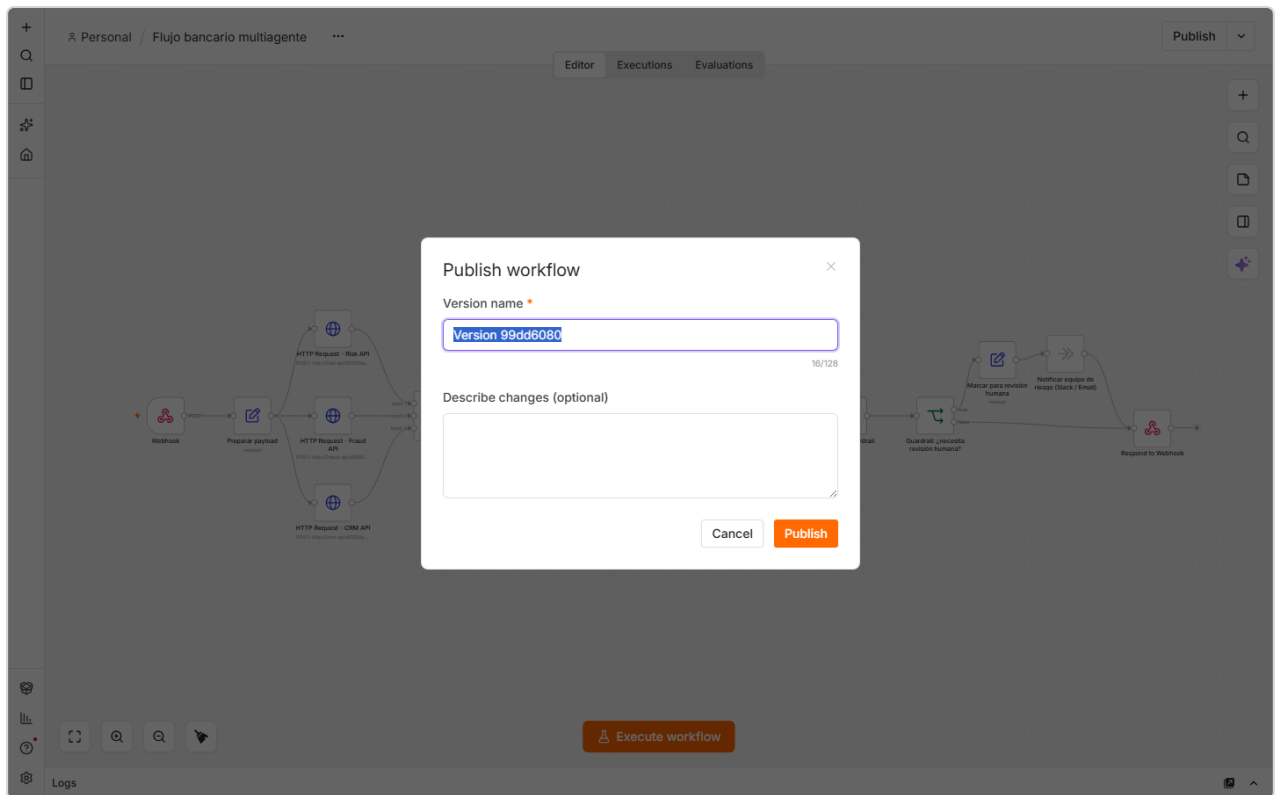
15. Publicar y probar con el script

La URL de prueba (/webhook-test/...) solo funciona mientras el editor espera. Para que el flujo responda siempre en /webhook/solicitud-bancaria , **publicálo**:

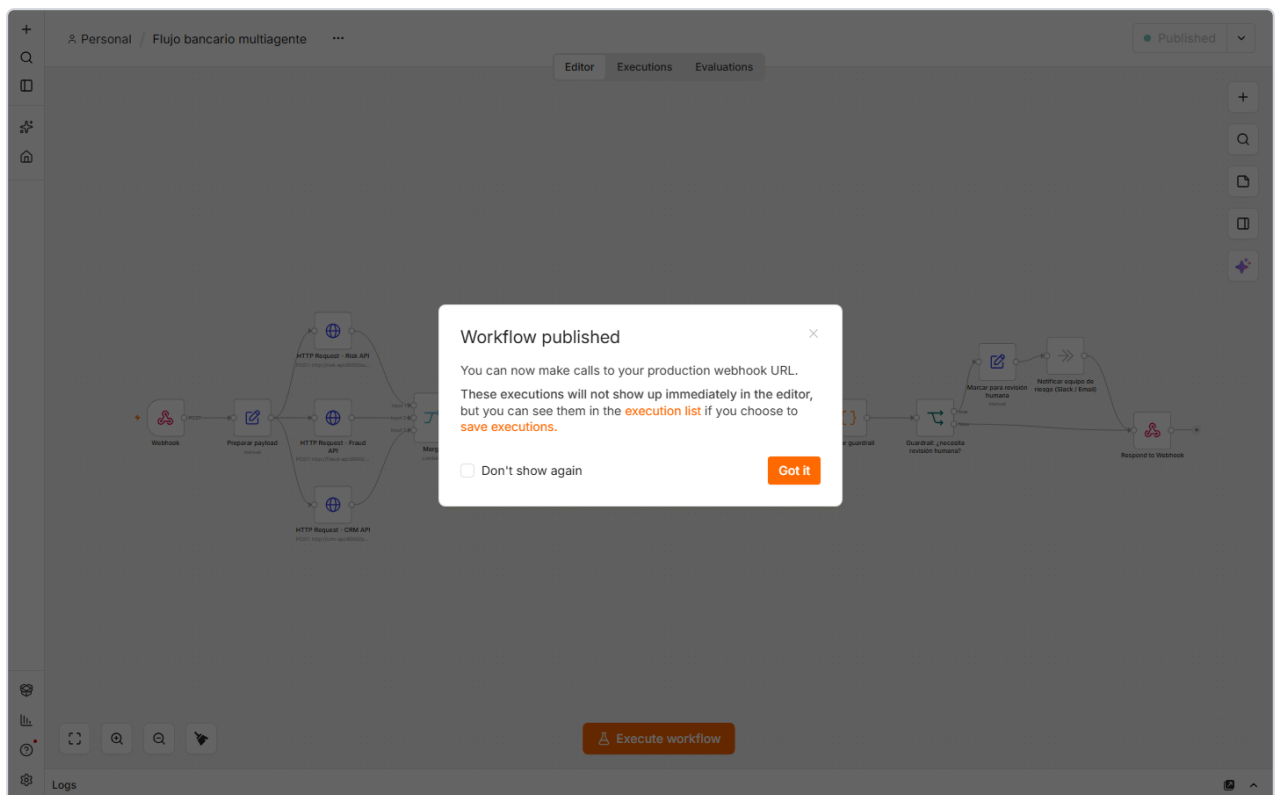
1. Haz clic en **Publish** (arriba a la derecha).



2. Deja el nombre de versión sugerido y haz clic en **Publish**.



3. Aparece *Workflow published* y el botón cambia a **Published**.



4. Envía los tres casos de ejemplo a la URL de producción:

```
.\scripts\test-flow.ps1
```

```
./scripts/test-flow.sh
```

Resultado esperado: el caso 1 y el 3 con la decisión del modelo (APROBADO , APROBADO_CON_RESTRICCIONES o RECHAZADO , según lo que decida) y el caso 2 con PENDIENTE_REVISION_HUMANA . Si cambias el flujo después de publicar, vuelve a hacer clic en **Publish** para que producción use la versión nueva.

16. Problemas comunes

Síntoma	Causa	Solución
Falta LLM_BASE_URL en el archivo .env (o LLM_API_KEY) al hacer <code>docker compose up -d</code>	No creaste .env o dejaste la línea vacía	Completa las dos líneas de .env (sección 1)
Cambié .env y no pasa nada	n8n lee .env solo al arrancar	Ejecuta otra vez <code>docker compose up -d</code>
Connect a model / Assistant responde Not Found	La llave del taller no es para el Asistente de n8n	No uses el Asistente; la llave solo la usa el nodo del paso 8
access to env vars denied en un nodo	Estás en otro n8n, no en el de este repositorio	Usa el n8n que levanta <code>docker compose up -d</code> (o el del codespace)
No Respond to Webhook node found in the workflow	Webhook con Respond = Using 'Respond to Webhook' Node sin ese nodo todavía	Usa <i>Immediately</i> mientras construyes (sección 4) y cámbialo en el paso 15
The requested webhook "solicitud-bancaria" is not registered (URL de prueba)	El editor no estaba esperando	Clic en <i>Listen for test event</i> o <i>Execute workflow</i> y reenvía; cada clic atiende una solicitud
Lo mismo, pero con /webhook/ (sin -test)	El flujo no está publicado	Sección 15: Publish
Fraud o CRM fallan con error 422 o campos vacíos	Los HTTP Request están encadenados en vez de en paralelo	Los tres deben salir de <i>Preparar payload</i> (sección 6)
Los HTTP Request reciben undefined	En <i>Preparar payload</i> los campos quedaron como <code>body.customer_id</code>	Renómbralos sin body. (sección 5)
getaddrinfo ENOTFOUND risk-api o ECONNREFUSED	Servicios caídos, o URL con localhost	<code>docker compose ps</code> ; usa <code>http://risk-api:8000/...</code>
Merge entrega datos incompletos	Number of Inputs distinto de 3 o entradas en otro orden	Risk → Input 1, Fraud → Input 2, CRM → Input 3
Referenced node doesn't exist en los pasos 10 u 11	El nodo del paso 7 o del paso 8 tiene otro nombre	Renómbralos exactamente <code>Combinar respuestas</code> y <code>Gemini Agent</code>
Gemini Agent: [ERROR: not accessible via UI...] bajo la URL o el header	Vista previa de \$env en el editor	Es normal; ejecuta el nodo
Gemini Agent: 401	Llave mal copiada, vencida o revocada	Corrige LLM_API_KEY en .env y ejecuta <code>docker compose up -d</code>
Gemini Agent: 429	Superaste el límite por minuto o el presupuesto de la llave	Espera un minuto; si dice presupuesto, avisa al organizador
Gemini Agent: 404	LLM_BASE_URL incorrecta	Debe ser solo el dominio, sin rutas; luego <code>docker compose up -d</code>

Síntoma	Causa	Solución
La respuesta dice " <i>Solicitud bloqueada...</i> "	La llave del taller solo admite peticiones del flujo bancario	Revisa que el JSON del paso 8 sea el de la guía
Un nodo Code o un campo JSON se ve con errores en rojo (<i>SyntaxError, Unexpected token, Invalid or unexpected token</i>)	El código se copió desde el PDF y quedó con saltos de línea partidos o caracteres invisibles	Bórralo y pégalo de nuevo desde <code>snippets/</code> o desde la versión web de la guía
Parsear respuesta: <i>Unexpected token</i>	El modelo respondió algo que no es JSON	Vuelve a ejecutar; si se repite, revisa que el JSON del paso 8 esté completo
<code>port is already allocated</code> (opción A)	El puerto 5678 está ocupado	Define <code>N8N_PORT=5679</code> en <code>.env</code> , abre http://localhost:5679 y usa <code>BASE_URL=http://localhost:5679</code> con los scripts
Codespaces: <i>Connection lost</i> permanente en el editor	El codespace se creó antes de la corrección de <code>.devcontainer/</code> (el navegador entra por <code>localhost</code> y n8n espera el dominio <code>app.github.dev</code>)	<code>git pull</code> , luego F1 → Codespaces: Rebuild Container (o crea un codespace nuevo); <code>docker compose ps</code> debe mostrar <code>n8n-proxy</code>
Codespaces: <i>Connection lost</i> un segundo y desaparece	El reenvío de puertos de GitHub cerró la conexión en vivo	Nada: n8n reconecta solo y no se pierde el flujo ni la ejecución
La respuesta trae datos del correo y no la decisión	Se usó <i>Send Email</i> en el paso 14	Usa <i>No Operation</i> (sección 12)